

Chapter 2

Instructions: Language of the Computer

Levels of Representation

High Level Language
Program (e.g., C)

```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

Current Focus

Compiler

Assembly Language Program
(e.g., RISC-V)

```
lw    t0, 0(s2)  
lw    t1, 4(s2)  
sw    t1, 0(s2)  
sw    t0, 4(s2)
```

Anything can be represented
as a *number*,
i.e., data or instructions

Assembler

Machine Language Program
(RISC-V)

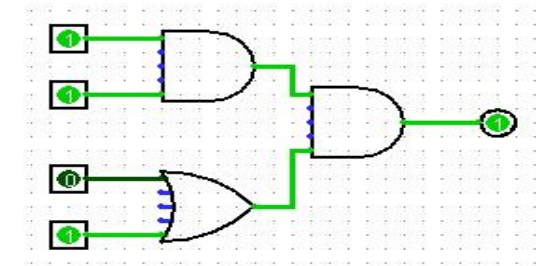
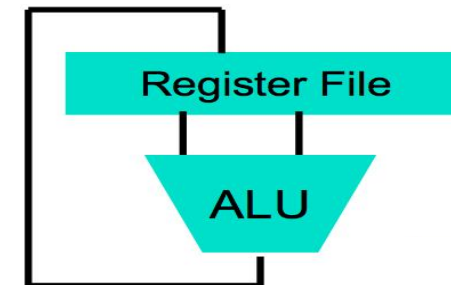
```
0000 1001 1100 0110 1010 1111 0101 1000  
1010 1111 0101 1000 0000 1001 1100 0110  
1100 0110 1010 1111 0101 1000 0000 1001  
0101 1000 0000 1001 1100 0110 1010 1111
```

Machine Interpretation

Hardware Architecture Description
(e.g., block diagrams)

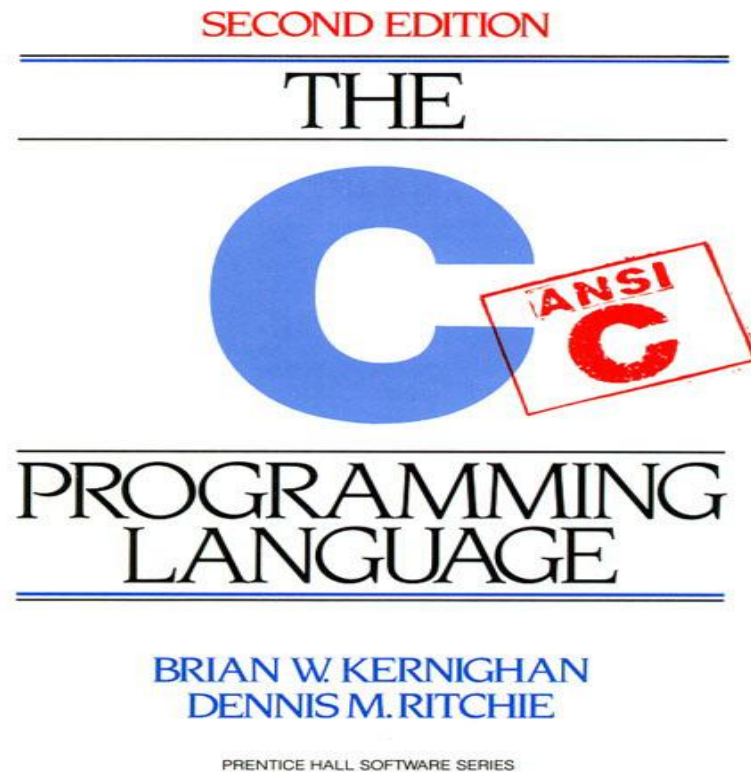
Architecture
Implementation

Logic Circuit Description
(Circuit Schematic Diagrams)



Introduction to C

“The Universal Assembly Language”



- Languages used in lab:
 - C
 - Assembly
- Prior programming experience helpful, e.g.
 - Java
 - C++

Hello World

C

```
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0;
}
```

Java

```
public class L02_HelloWorld {
    public static void main(String args[]) {
        System.out.println("Hello World!");
    }
}
```

Compilation & Running

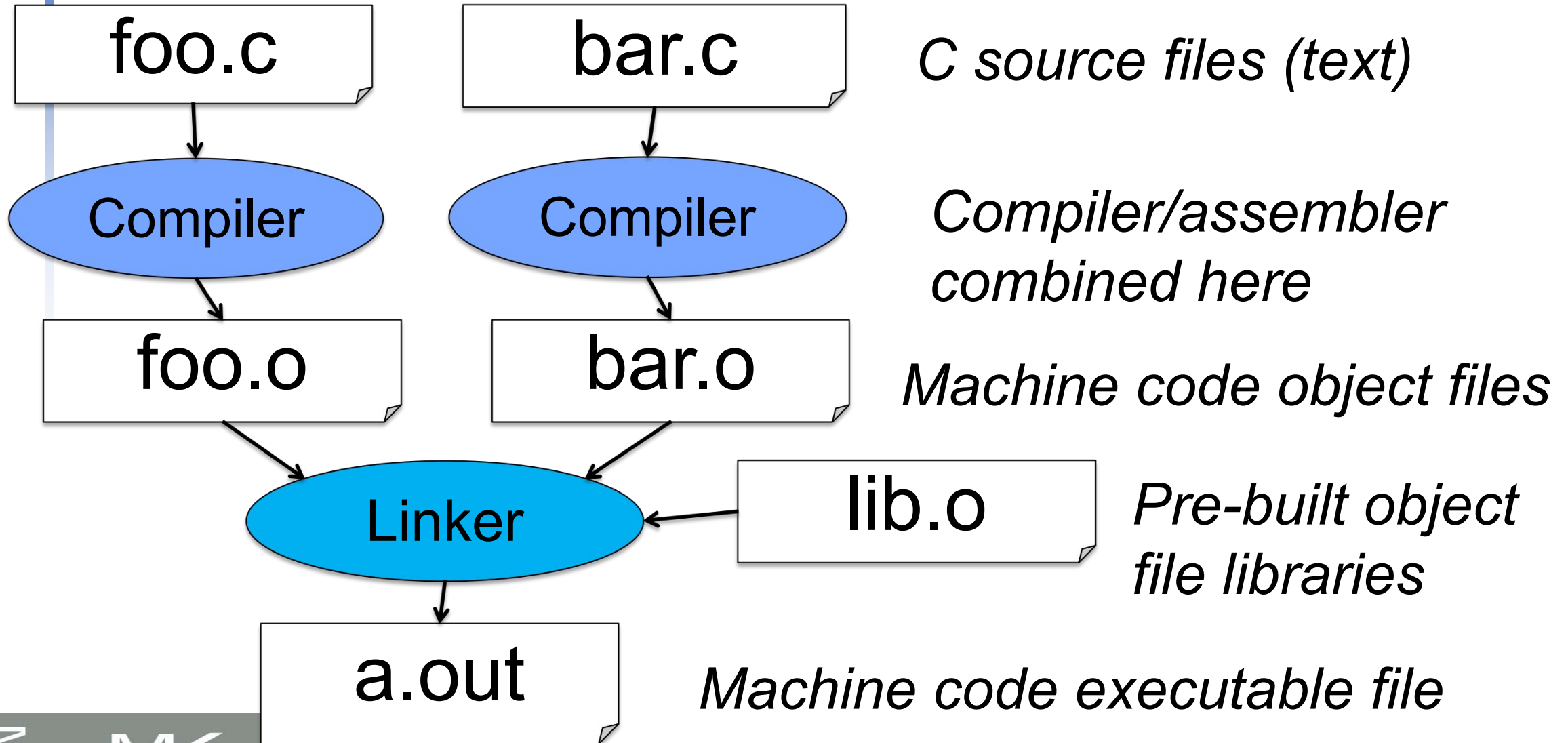
```
$ gcc HelloWorld.c
```

```
$ ./a.out
```

```
Hello World!
```

```
$
```

C Compilation Simplified Overview (more later in course)



Compilation versus Interpretation

C (compiled)

- Compiler (& linker) translates source into machine language
- Machine language program is loaded by OS and directly executed by the hardware

Python (interpreted)

- Interpreter is written in some high-level language (e.g. C) and translated into machine language
- Loaded by OS and directly executed by processor
- Interpreter reads source code (e.g. Python) and “interprets” it

Java “Byte Code”

- Java compiler (javac) translates source to “byte code”
- “Byte code” is a particular assembly language
 - Just like i86, RISC-V, ARM, ...
 - Can be directly executed by appropriate machine
 - implementations exist(ed), not commercially successful
 - More typically, “byte code” is
 - interpreted on target machine (e.g. i86) by java program
 - compiled to target machine code (e.g. by JIT)
 - Program runs on any computer with a “byte code” interpreter (more or less)

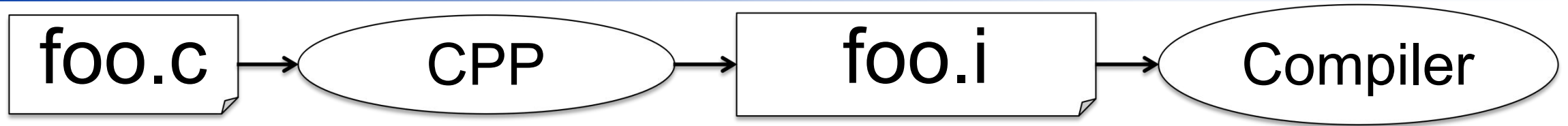
Compilation

- Excellent run-time performance:
 - Much faster than interpreted code (e.g. Python)
 - Usually faster than Java (even with JIT)
- Note: Computers only run machine code
 - Compiled application program, or
 - Interpreter (that interprets source code)

Compilation: Disadvantages

- Compiled files, including the executable, are
 - **architecture-specific**, depending on processor type
 - e.g., x86, MIPS, RISC-V, ARM
 - **and the operating system**
 - e.g., Windows vs. Linux
- Executable **must be rebuilt** on each new system
 - i.e., “porting your code” to a new architecture
- “Change → Compile → Run [repeat]” iteration cycle can be slow during development
 - Recompile only parts of program that have changed
 - Tools (e.g. make) automate this

C Pre-Processor (CPP)



- C source files pass through macro processor, CPP, before compilation
- CPP replaces comments with a single space
- CPP commands begin with “#”
- `#include "file.h" /* Inserts file.h */`
- `#include <stdio.h> /* Loads from standard loc */`
- `#define M_PI (3.14159) /* Define constant */`
- `#if/#endif /* Conditional inclusion of text */`
- Use `–save-temps` option to gcc to see result of preprocessing
- Full documentation at: <http://gcc.gnu.org/onlinedocs/cpp/>

Integers: Python vs. Java vs. C

Language	sizeof(int)
Python	≥ 32 bits (plain ints), infinite (long ints)
Java	32 bits
C	Depends on computer; 16 or 32 or 64

- C: `int`
 - integer type that target processor works with most efficiently
- Only guarantee:
 - $\text{sizeof}(\text{long long}) \geq \text{sizeof}(\text{long}) \geq \text{sizeof}(\text{int}) \geq \text{sizeof}(\text{short})$
 - Also, `short` ≥ 16 bits, `long` ≥ 32 bits
 - All could be 64 bits
- Impacts portability between architectures

Variable Sizes: Machine Dependent!

```
#include <stdio.h>
int main(void) {
    printf("sizeof ... (bytes)\n");
    printf("char:          %lu\n",
           sizeof(char));
    printf("short:         %lu\n",
           sizeof(short));
    printf("int:            %lu\n",
           sizeof(int));
    printf("unsigned int: %lu\n",
           sizeof(unsigned int));
    printf("long:          %lu\n",
           sizeof(long));
    printf("long long:     %lu\n",
           sizeof(long long));
    printf("float:         %lu\n",
           sizeof(float));
    printf("double:        %lu\n",
           sizeof(double));
}
```

sizeof ... (bytes)

char: 1

short: 2

int: 4

unsigned int: 4

long: 8

long long: 8

float: 4

double: 8

Boolean

- No boolean datatype in C

- Declare if you wish:

```
typedef int boolean;  
const boolean false = 0;  
const boolean true = 1;
```

- What evaluates to FALSE in C?

- 0 (integer)
 - NULL (a special kind of *pointer*: more on this later)

- What evaluates to TRUE in C?

- Anything that isn't false is true
 - Similar to Python:

only 0's or empty sequences are false, everything else is true!

Uninitialized Variables

Code

```
#include <stdio.h>
#include <stdlib.h>

void undefined_local() {
    int x; /* undefined */
    printf("x = %d\n", x);
}

void some_calc(int a) {
    a = a%2 ? rand() : -a;
}

int main(void) {
    for (int i=0; i<5; i++) {
        some_calc(i*i);
        undefined_local();
    }
}
```

Output

```
$ gcc test.c

$ ./a.out
x = 0
x = 16807
x = -4
x = 282475249
x = -16
```

Computer Memory

```
int a;
```

```
a = -85;
```

```
printf("%d", a);
```

Type	Name	Addr	Value
		...	
		108	
		107	
		106	
		105	
		104	
		103	
		102	
		101	
		100	
		...	

Do not confuse memory address and value.
Nor **a street address** with **the person** living there.

Pointers

- C speak for “memory addresses”

- Notation

`int *x;      // variable x is an address to an int`

`int y = 9;   // y is an int`

`x = &y;      // assign address of y to x`

`// “address operator”`

`int z = *x;   // assign what x is pointing to to z`

`// “dereference operator”`

`*x = -7;      // assign -7 to what x is pointing to`

What are the values of x, y, z?

Type	Name	Addr	Value
		...	
		108	
		107	
		106	
		105	
		104	
		103	
		102	
		101	
		100	
		...	

Pointer Type

- **Pointers have types**, like other variables
 - “type of object” the pointer is “pointing to”
- **Examples:**
 - `int *pi; // pointer to int`
 - `double *pd; // pointer to double`
 - `char *pc; // pointer to char`

Generic Pointer (void *)

- Generic pointer

- Points to any object (int, double, ...)
- Does not “know” type of object it references
(e.g. compiler does not know)

- Example:

- ```
void *vp; // vp holds an address to
 // object of “arbitrary” type
```

- Applications

- Generic functions e.g. to allocate memory
- `malloc, free`
  - accept and return pointers of any type
  - see next lecture

# Pointer to struct

```
// type declaration
typedef struct { int x, y; } Point;

// declare (and initialize) Point "object"
Point pt = { 0, 5 };

// declare (and initialize) pointer to Point
Point *pt_ptr = &pt;

// access elements
(*pt_ptr).x = (*pt_ptr).y;

// alternative syntax
pp->x = pp->y;
```

# Your Turn!

```
#include <stdio.h>

int main(void) {
 int a = 3, b = -7;
 int *pa = &a, *pb = &b;
 *pb = 5;
 if (*pb > *pa) a = *pa - b;
 printf("a=%d b=%d\n", a, b);
}
```

| Answer | a  | b  |
|--------|----|----|
| RED    | 3  | -7 |
| GREEN  | 4  | 5  |
| ORANGE | -4 | 5  |
| YELLOW | -2 | 5  |

| Type | Name | Addr | Value |
|------|------|------|-------|
|      |      | ...  |       |
|      |      | 108  |       |
|      |      | 107  |       |
|      |      | 106  |       |
|      |      | 105  |       |
|      |      | 104  |       |
|      |      | 103  |       |
|      |      | 102  |       |
|      |      | 101  |       |
|      |      | 100  |       |
|      |      | ...  |       |

# C Arrays

- Declaration:
  - `// allocate space  
// unknown content  
int a[5];`
  - `// allocate & initialize  
int b[] = { 3, 2, 1 };`
- Element access:
  - `b[1];`
  - `a[2] = 7;`
- Index of first element: 0

| Type | Name | Addr | Value |
|------|------|------|-------|
|      |      | ...  |       |
|      |      | 108  |       |
|      |      | 107  |       |
|      |      | 106  |       |
|      |      | 105  |       |
|      |      | 104  |       |
|      |      | 103  |       |
|      |      | 102  |       |
|      |      | 101  |       |
|      |      | 100  |       |
|      |      | ...  |       |

# Beware: no array bound checking!

```
#include <stdio.h>

int main(void) {
 int a[] = { 1, 2, 3 };

 for (int i=0; i<4; i++)
 printf("a[%d] = %d\n", i, a[i]);
}
```

Output:

|      |   |             |
|------|---|-------------|
| a[0] | = | 1           |
| a[1] | = | 2           |
| a[2] | = | 3           |
| a[3] | = | -1870523725 |

Often the result is much worse:

- erratic behavior
- segmentation fault, etc.
- C does not know array length!
- Pass as argument into functions

# Use Constants, Not Literals

- Assign size to constant

- Bad pattern

- ```
int i, ar[10];  
for(i = 0; i < 10; i++){ ... }
```

- Better pattern

- ```
const int ARRAY_SIZE = 10;
int i, a[ARRAY_SIZE];
for(i = 0; i < ARRAY_SIZE; i++){ ... }
```

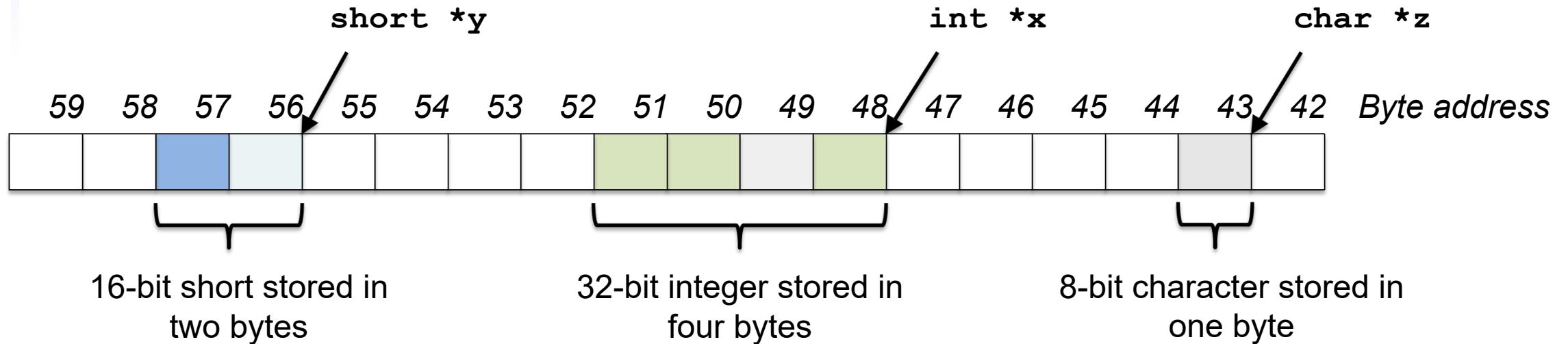
- “Single source of truth”

- Avoiding maintaining two copies of the number 10
  - And the chance of changing only one
  - DRY: “Don’t Repeat Yourself”



# Pointing to Different Size Objects

- Modern machines are “byte-addressable”
  - Hardware’s memory composed of 8-bit storage cells,  
each byte in main memory has a unique address
- Type declaration tells compiler how many bytes to fetch on each access through pointer
  - E.g., 32-bit integer stored in 4 consecutive 8-bit bytes



# Pointer Arithmetic - int

```
#include <stdio.h>
```

```
int main(void) {
 char c[] = { 'a', 'b' };
 char *pc = c;
 pc++;
 printf("*pc=%c\n c=%p\npc=%p\npc-c=%ld\n",
 *pc, c, pc, pc-c);

 int i[] = { 10, 20 };
 int *pi = i;
 pi++;
 printf("*pi=%d\n i=%p\npi=%p\npi-i=%ld\n",
 *pi, i, pi, pi-i);
}
```

```
*pi = 20
i = 0x7fff50f54b40
pi = 0x7fff50f54b44
pi-i = 1
```

| Type | Name | Byte Addr | Value |
|------|------|-----------|-------|
|      |      | ...       |       |
|      |      | 108       |       |
|      |      | 107       |       |
|      |      | 106       |       |
|      |      | 105       |       |
|      |      | 104       |       |
|      |      | 103       |       |
|      |      | 102       |       |
|      |      | 101       |       |
|      |      | 100       |       |
|      |      | ...       |       |

# Array Name / Pointer Duality

- Array variable is a “pointer” to the first (0<sup>th</sup>) element
- Can use pointers to access array elements
  - `char *pstr` and `char astr[]` are nearly identical declarations
  - Differ in subtle ways: `astr++` is illegal
- Consequences:
  - `astr` is an array variable, but works like a pointer
  - `astr[0]` is the same as `*astr`
  - `astr[2]` is the same as `*(astr+2)`
  - Can use pointer arithmetic to access array elements

# C Strings

- C strings are **null-terminated** character arrays
  - `char s[] = "abc";`

| Type | Name | Byte Addr | Value |
|------|------|-----------|-------|
|      |      | ...       |       |
|      |      | 108       |       |
|      |      | 107       |       |
|      |      | 106       |       |
|      |      | 105       |       |
|      |      | 104       |       |
|      |      | 103       |       |
|      |      | 102       |       |
|      |      | 101       |       |
|      |      | 100       |       |
|      |      | ...       |       |

# String Example

```
#include <stdio.h>
```

```
int slen(char s[]) {
 int n = 0;
 while (s[n] != 0) n++;
 return n;
}
```

```
int main(void) {
 char str[] = "abc";
 printf("str = %s, length = %d\n", str, slen(str));
}
```

**Output:** str = abc, length = 3

# Arguments in main ()

- To get arguments to the main function, use:
  - `int main(int argc, char *argv[])`
  - `argc` is the *number* of strings on the command line
  - `argv` is a *pointer* to an array containing the arguments as strings

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {
 for (int i=0; i<argc; i++)
 printf("arg[%d] = %s\n", i, argv[i]);
}
```

# What's wrong with this Code?

```
#include <stdio.h>
#include <math.h>

int *f() {
 int x = 5;
 return &x;
}

int main(void) {
 int *a = f();
 // ... some calculations ...
 double d = cos(1.57);
 // ... now use *a ...
 printf("a = %d\n", *a);
}
```

## Output:

a = -1085663214

- `*a` is a pointer to a local variable
  - allocated on the stack
  - “deallocated” when `f()` returns
  - stack reused by other functions
    - e.g. `cos`
    - which overwrite whatever was there before
  - `*a` points to “garbage”
- Obscure errors
  - depend on what other functions are called after `f()` returns
  - assignments to `*a` corrupt the stack and can result in even more bizarre behavior than this example
  - errors can be difficult to reproduce

# Example

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {
 for (int i=0; i<argc; i++)
 printf("arg[%d] = %s\n", i, argv[i]);
}
```

**UNIX:**

```
$ gcc -o ex Argc.c
```

```
$./ex -g a "d e f"
```

```
arg[0] = ./ex
```

```
arg[1] = -g
```

```
arg[2] = a
```

```
arg[3] = d e f
```



# Functions in C

```
int number_of_people() {
 return 3;
}
```

```
void news() {
 printf("no news");
}
```

```
int sum(int x, int y) {
 return x + y;
}
```

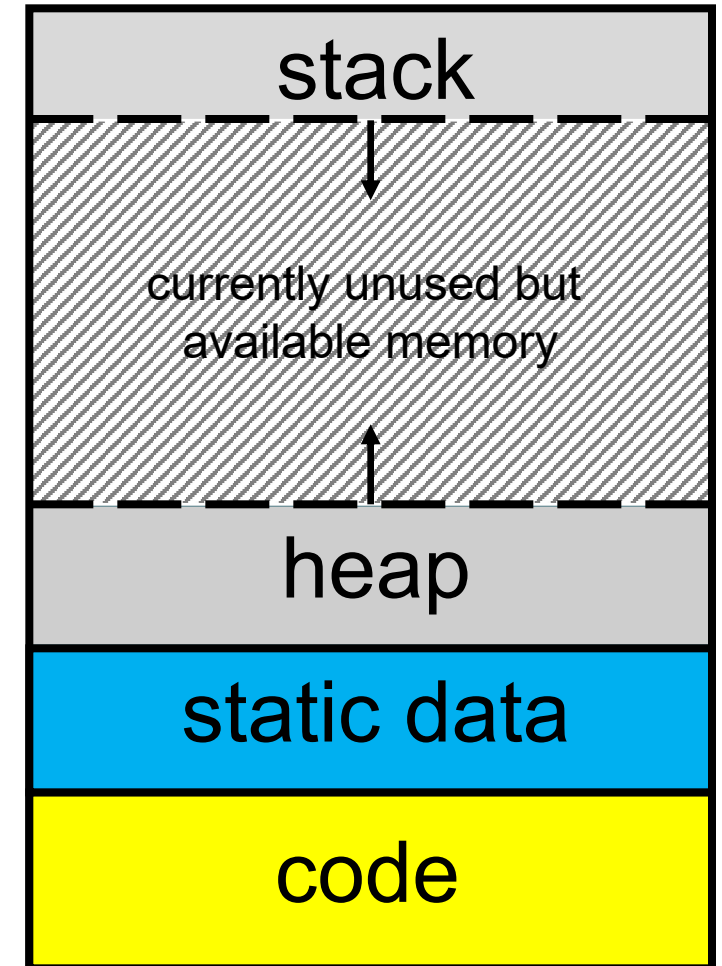
- Like Java
- Declare return & argument types
- **void** for no value returned
- Functions **MUST** be declared before they are used

# C Memory Management

- How does the C compiler determine where to put code and data in the machine's **main memory**?
- How can we create dynamically sized objects?
  - E.g. array of variable size depending on requirements

~ FFFF FFFF<sub>hex</sub>

Sample Layout  
(32-bit addresses)



~ 0000 0000<sub>hex</sub>

# C Memory Management: Code

## ■ Code

- Loaded when program starts
- Does not change

```
int static_data = 55;
```

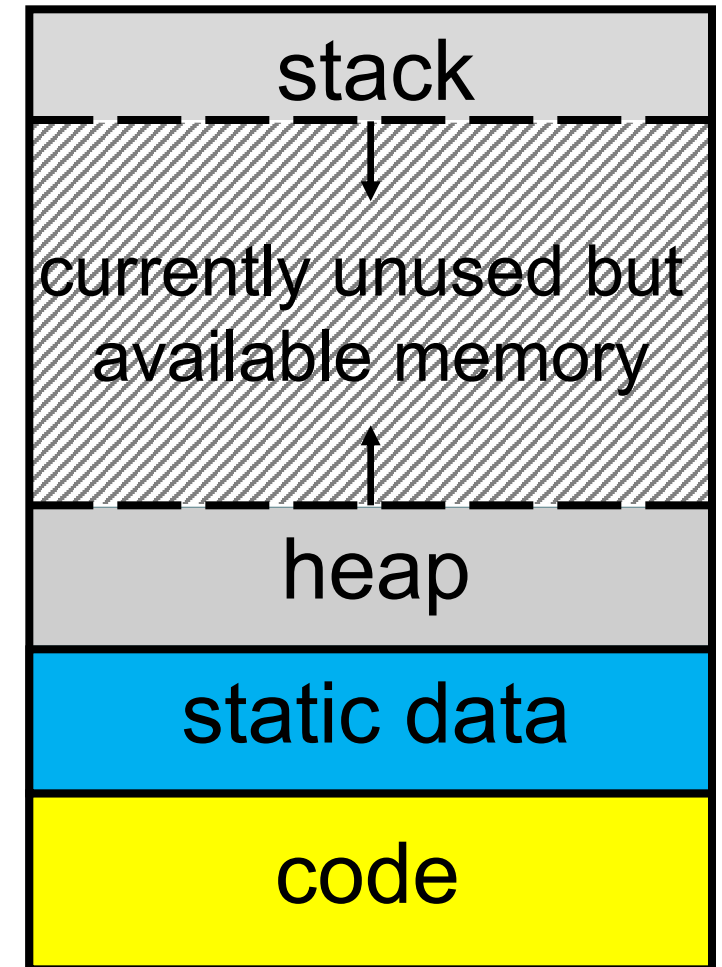
```
void f(char c) {
 int local_data = 4;
 int *heap_data
 = malloc(50*sizeof(int));
}
```

```
int main(void) {
 int local_data;
 char more_local_data = 'X';
 f(more_local_data);
}
```

~ FFFF FFFF<sub>hex</sub>

~ 0000 0000<sub>hex</sub>

Sample Layout  
(32-bit addresses)



# C Memory Management: Static Data

## ■ Static Data

- Loaded when program starts
- Can be modified
- Size is fixed

```
int static_data = 55;
```

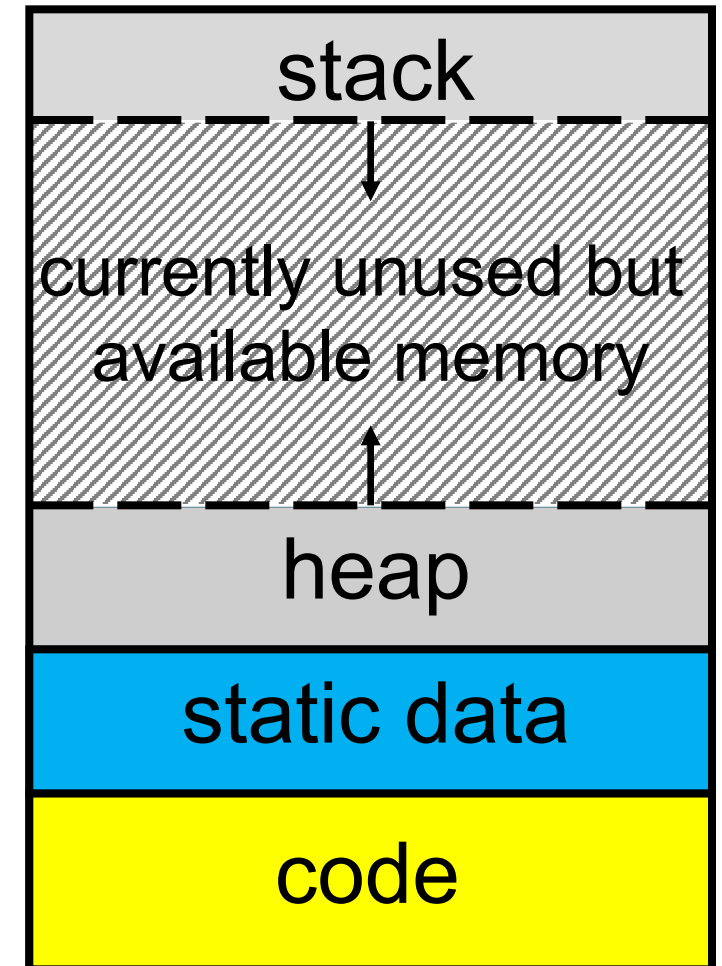
```
void f(char c) {
 int local_data = 4;
 int *heap_data
 = malloc(50*sizeof(int));
}
```

```
int main(void) {
 int local_data;
 char more_local_data = 'X';
 f(more_local_data);
}
```

~ FFFF FFFF<sub>hex</sub>

~ 0000 0000<sub>hex</sub>

Sample Layout  
(32-bit addresses)



# C Memory Management: Stack

## Stack

- Local variables & arguments inside functions
- Allocated when function is called
- Stack usually grows **downward**

```
int static_data = 55;
```

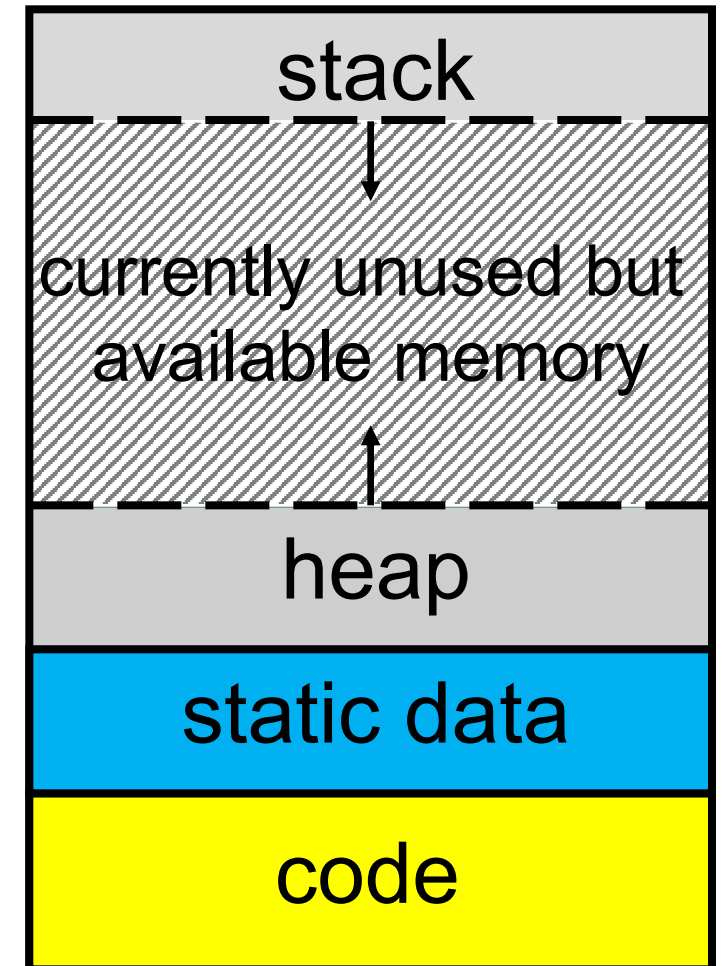
```
void f(char c) {
 int local_data = 4;
 int *heap_data
 = malloc(50*sizeof(int));
}
```

```
int main(void) {
 int local_data;
 char more_local_data = 'X';
 f(more_local_data);
}
```

~ FFFF FFFF<sub>hex</sub>

~ 0000 0000<sub>hex</sub>

Sample Layout  
(32-bit addresses)



# C Memory Management: Heap

## ■ Heap

- Space for dynamic data
- Allocated and freed by program as needed

```
int static_data = 55;
```

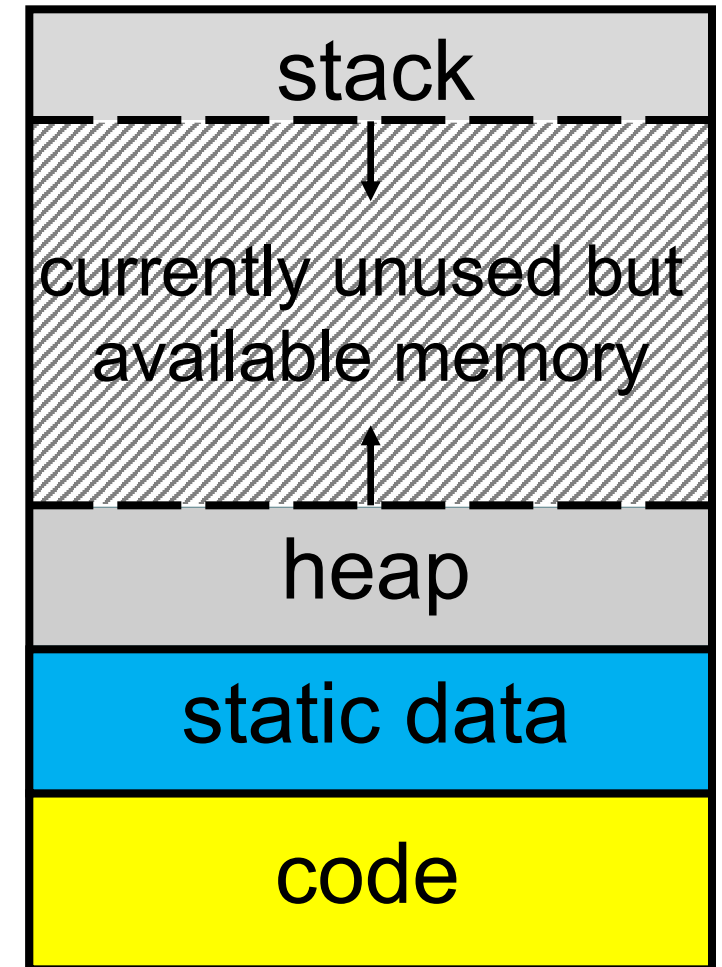
```
void f(char c) {
 int local_data = 4;
 int *heap_data
 = malloc(50*sizeof(int));
}
```

```
int main(void) {
 int local_data;
 char more_local_data = 'X';
 f(more_local_data);
}
```

~ FFFF FFFF<sub>hex</sub>

~ 0000 0000<sub>hex</sub>

Sample Layout  
(32-bit addresses)



# Stack

```
void a() {
 int a_local = 0;
 b(a_local);
}
```

```
int b(int arg) {
 int b_local = 5;
 return 2*arg + b_local;
}
```

```
int main(void) {
 a();
 b(7);
}
```

Stack Pointer

Stack Pointer

Stack Pointer

Stack Pointer

**Frame main**

**Frame a**

int a\_local;

**Frame b**

int arg;

int b\_local;

# Stack

- Every time a function is called, a new frame is allocated
- When the function returns, the frame is deallocated
- Stack frame contains
  - Function arguments
  - Local variables
  - Return address (who called me?)
- Stack uses contiguous blocks of memory
  - Stack pointer indicates current level of stack
- Stack management is transparent to C programmer
  - We'll see details when we program assembly language



# Your Turn ...

```
#include <stdio.h>
#include <stdlib.h>

int x = 2;

int foo(int n) {
 int y;
 if (n <= 0) {
 printf("End case!\n");
 return 0;
 } else {
 y = n + foo(n - x);
 return y;
 }
}

int main(void) {
 foo(10);
}
```

Right after the `printf` executes but before the `return 0`, how many copies of `x` and `y` are allocated in memory?

| Answer | #x | #y |
|--------|----|----|
| RED    | 1  | 1  |
| GREEN  | 1  | 6  |
| ORANGE | 1  | 5  |
| YELLOW | 6  | 6  |

# What's wrong with this Code?

```
#include <stdio.h>
#include <math.h>

int *f() {
 int x = 5;
 return &x;
}

int main(void) {
 int *a = f();
 // ... some calculations ...
 double d = cos(1.57);
 // ... now use *a ...
 printf("a = %d\n", *a);
}
```

## Output:

a = -1085663214

- `*a` is a pointer to a local variable
  - allocated on the stack
  - “deallocated” when `f()` returns
  - stack reused by other functions
    - e.g. `cos`
    - which overwrite whatever was there before
  - `*a` points to “garbage”
- **Obscure errors**
  - depend on what other functions are called after `f()` returns
  - assignments to `*a` corrupt the stack and can result in even more bizarre behavior than this example
  - errors can be difficult to reproduce

# Managing the Heap

C functions for heap management:

- **malloc()** allocate a block of uninitialized memory
- **calloc()** allocate a block of zeroed memory
- **free()** free previously allocated block of memory
- **realloc()** change size of previously allocated block

- **Beware:**  
*previously allocated contents might move!*

# Malloc()

- `void *malloc(size_t n):`
  - Allocate a block of uninitialized memory
  - `n` is an integer, indicating size of requested memory block in bytes
  - `size_t` is an unsigned integer type big enough to “count” memory bytes
  - Returns `void*` pointer to block
  - `NULL` return indicates no more memory

Example: `#include <stdlib.h>`

```
int main(void) {
 // array of 50 ints ...
 int *ip = (int*)malloc(50*sizeof(int));

 // typecast is optional
 double *dp = malloc(1000*sizeof(double));
}
```

# What's wrong with this Code?

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
 const int SZ = 10;
 int *p = malloc(SZ*sizeof(int));
 int *end = &p[SZ];
 while (p < end) *p++ = 0;
 free(p);
}
```

```
$ gcc FreeBug.c; ./a.out
```

```
a.out(23562,0x7fff78748000) malloc:
```

```
*** error for object 0x7fdcb3403168:
```

```
 pointer being freed was not allocated
```

```
*** set a breakpoint in malloc_error_break to debug
```

```
Abort trap: 6
```

# free()

- `void free(void *p):`
  - Release memory allocated by `malloc()`
  - ***p must contain address originally returned by malloc()***

- Example:

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
 double *dp = malloc(100*sizeof(double));
 // ... do something with *dp ...
 // ...
 // Give memory back when done!
 free(dp); // free knows it's 100 doubles
}
```



# Fix

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
 const int SZ = 10;
 int *array = malloc(SZ*sizeof(int));
 for (int *p = array; p<&array[SZ];)
 *p++ = 0;
 free(array);
}
```

- Do not modify return value from `malloc`
  - You'll need it to call `free`!

# Why Call free()?

- Recycle **no-longer-used memory** to avoid running out
- Two common approaches:
  - malloc/free (Explicit memory management)
    - C, C++, ...
    - "manually" take care of releasing memory
    - Requires some planning: how do I know that memory is no longer used? What if I forget to release?
    - Drawback: potential bugs
      - memory leaks (forgot to call free)
      - corrupted memory (accessing memory that is now longer "owned")
  - garbage collector (Automatic memory management)
    - Java, Python, ...
    - No-longer-used memory is free'd automatically
    - Drawbacks:
      - performance hit
      - unpredictable:
        - what if garbage collector starts when a self-driving car enters a turn at 100mph?



# Out of Memory

- Insufficient free memory: `malloc()` returns `NULL`

```
int main(void) {
 const int G = 1024*1024*1024;
 for (int n=0; ;n++) {
 char *p = malloc(G*sizeof(char));
 if (p == NULL) {
 fprintf(stderr,
 "failed to allocate > %g TiBytes\n", n/1000.0);
 return 1; // abort program
 }
 // no free, keep allocating until out of memory
 }
}
```

```
$ gcc OutOfMemory.c; ./a.out
```

```
failed to allocate > 131.064 TiBytes
```

# Example: Dynamically Allocated Tree

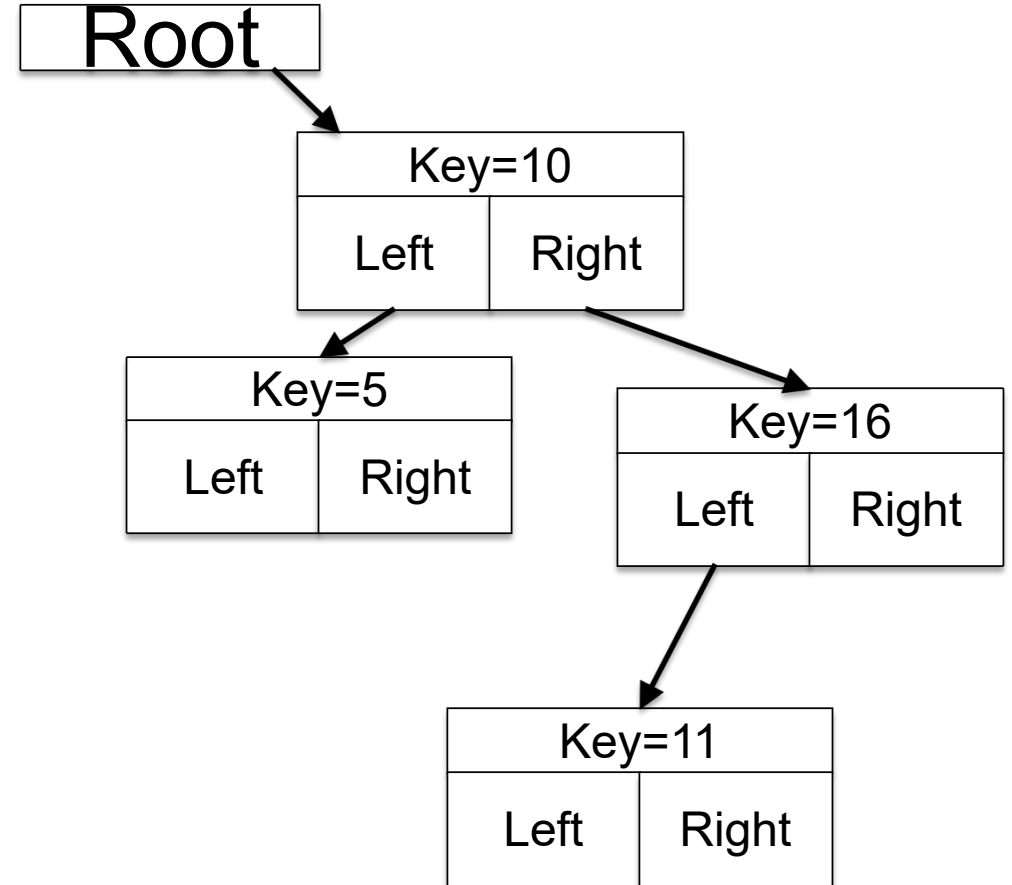
```
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
 int key;
 struct node *left;
 struct node *right;
} Node;

Node *create_node(int key, Node *left, Node *right) {
 Node *np;
 if ((np = malloc(sizeof(Node))) == NULL) {
 printf("Out of Memory!\n"); exit(1);
 } else {
 np->key = key;
 np->left = left;
 np->right = right;
 return np;
 }
}

void insert(int key, Node **tree) {
 if ((*tree) == NULL) {
 (*tree) = create_node(key, NULL, NULL); return;
 }
 if (key <= (*tree)->key) insert(key, &((*tree)->left));
 else insert(key, &((*tree)->right));
}

int main(void) {
 Node *root = NULL;
 insert(10, &root);
 insert(16, &root);
 insert(5, &root);
 insert(11, &root);
}
```



# malloc and free are buddies!

## malloc **and** free

- If you call **malloc** somewhere, you'll need to call **free** on the result (or accept having memory leak)

- E.g.

```
int *p = malloc(...);
// potentially very
// complicated code
// when done, call:
free(p);
```

## **no** malloc, **no** free

- If you have not called **malloc**, do not call **free**!

- E.g.

```
int x;
int *p = &x;
// do whatever with p,
// but do not call free!
int a[5];
// do whatever with a, but
// do not call free(a)!
```

# Observations

- Code, Static storage are easy:
  - they never grow or shrink
  - taken care of by OS
- Stack space is relatively easy:
  - stack frames are created and destroyed in last-in, first-out (LIFO) order
  - transparent to programmer
  - but don't hold onto it (with pointer) after function returns!
- *Managing the heap is the programmer's task:*
  - memory can be allocated / deallocated at any time
  - how tell / ensure that a block of memory is no longer used *anywhere* in the program?
  - requires planning before coding ...

# Break!



# Levels of Representation/Interpretation

High Level Language  
Program (e.g., C)

*Compiler*

**Assembly Language Program  
(e.g., RISC-V)**

*Assembler*

Machine Language Program  
(RISC-V)

*Machine Interpretation*

Hardware Architecture Description  
(e.g., block diagrams)

*Architecture  
Implementation*

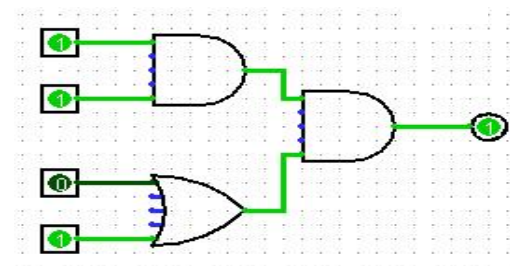
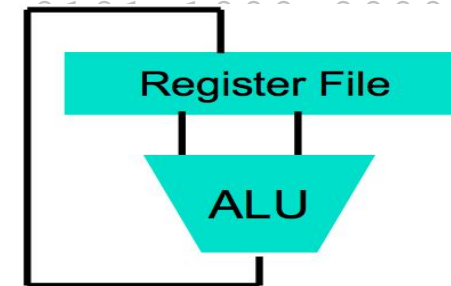
Logic Circuit Description  
(Circuit Schematic Diagrams)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw t0, 0(s2)
lw t1, 4(s2)
sw t1, 0(s2)
sw t0, 4(s2)
```

*Current Focus*

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 1100 0110 1010 1111 1001 1100 0110 1010 1111
```



# Instruction Set Architecture (ISA)

- Job of a CPU (*Central Processing Unit*, aka *Core*): execute *instructions*
- Instructions: CPU's primitives operations
  - Like a sentence: operations (verbs) applied to operands (objects) processed in sequence ...
  - With additional operations to change the sequence
- CPUs belong to “families,” each implementing its own set of instructions

# Instruction Set

- CPU's particular set of instructions implements an *Instruction Set Architecture (ISA)*
  - Examples: ARM, Intel x86, MIPS, RISC-V, IBM/Motorola PowerPC (old Mac), Intel IA64, ...
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets



# Instruction Set Architectures

- Early trend: add more instructions to new CPUs for elaborate operations
  - VAX architecture had an instruction to multiply polynomials!
- RISC philosophy (Cocke IBM, Patterson UCB, Hennessy Stanford, 1980s) – *Reduced Instruction Set Computing*
  - Keep the instruction set small and simple, in order to build fast hardware
  - Let software do complicated operations by composing simpler ones

# The MIPS Instruction Set

- Used as the example throughout the book
- Stanford MIPS commercialized by MIPS Technologies ([www.mips.com](http://www.mips.com))
- Typical of many modern ISAs
  - See MIPS Reference Data tear-out card, and Appendixes B and E
- Similar ISAs have a large share of embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, ...

# What is RISC-V?



- Fifth generation of RISC design from UC Berkeley
- A high-quality, license-free, royalty-free RISC ISA specification
- Experiencing rapid uptake in both industry and academia
- Both proprietary and open-source core implementations
- Supported by growing shared software ecosystem
- Appropriate for all levels of computing system, from microcontrollers to supercomputers
  - 32-bit, 64-bit, and 128-bit variants (we're using 32-bit in class, textbook uses 64-bit)
- Standard maintained by non-profit RISC-V Foundation



# Foundation Members (60+)

Platinum:



Berkeley  
Architecture  
Research



Microsoft

Rambus

Cryptography Research



SiFive



SAMSUNG

DRAPER



Microsemi



NVIDIA



Western  
Digital



Gold, Silver, Auditors:



Blockstream



中科院计算所  
INSTITUTE OF COMPUTING TECHNOLOGY, CAS



Esperanto  
Technologies



MEDIATEK



INTRINSIX



ETH zürich



lowRISC

MINIMA  
PROCESSOR



Rumble



runtime.io



Syntacore  
Custom cores and tools

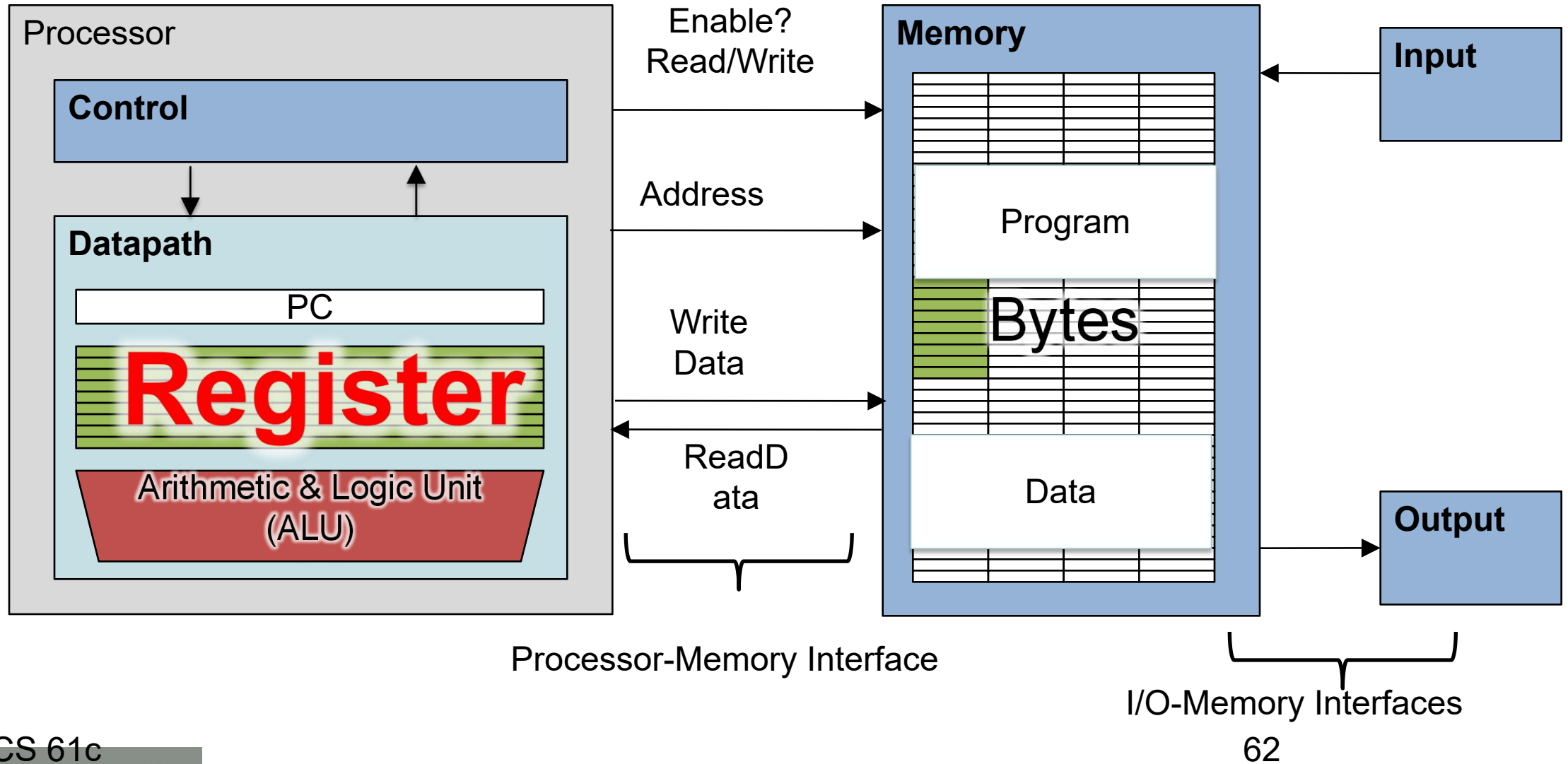


VectorBlox  
embedded supercomputing

# Assembly Variables: Registers

- Unlike HLL like C or Java, assembly does not have *variables* as you know and love them
  - More primitive, closer what simple hardware can directly support
- Assembly operands are objects called registers
  - Limited number of special places to hold values, built directly into the hardware
  - Operations can only be performed on these!
- Benefit: Since registers are directly in hardware, they are very fast (faster than 1 ns - light travels 1 foot in 1 ns!!! )

# Registers live inside the Processor



# Number of MIPS Registers

- Drawback: Since registers are in hardware, there are a limited number of them
  - Solution: MIPS code must be carefully written to efficiently use registers
- 32 registers in MIPS, referred to by number **\$0 – \$31**
  - Registers are also given symbolic names, described later
  - Why 32? **Smaller is faster, but too small is bad. Goldilocks principle** (“This porridge is too hot; This porridge is too cold; this porridge is just right”)
    - Each MIPS register is **32 bits wide**
  - Groups of 32 bits called a **word** in MIPS ISA
- **\$0** is a special register, **always holds value zero**
  - So really only 31 registers able to hold variable values



# MIPS Registers

|        |               |                                                                                                                                                                                      |
|--------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0      | <b>zero</b>   | Always returns 0                                                                                                                                                                     |
| 1      | <b>at</b>     | (assembly temporary) Reserved for use by assembly                                                                                                                                    |
| 2–3    | <b>v0, v1</b> | Value returned by subroutine                                                                                                                                                         |
| 4–7    | <b>a0–a3</b>  | (arguments) First few parameters for a subroutine                                                                                                                                    |
| 8–15   | <b>t0–t7</b>  | (temporaries) Subroutines can use without saving                                                                                                                                     |
| 24, 25 | <b>t8, t9</b> |                                                                                                                                                                                      |
| 16–23  | <b>s0–s7</b>  | Subroutine register variables; a subroutine that writes one of these must save the old value and restore it before it exits, so the <i>calling</i> routine sees the values preserved |
| 26, 27 | <b>k0, k1</b> | Reserved for use by interrupt/trap handler; may change under your feet                                                                                                               |
| 28     | <b>gp</b>     | Global pointer; some runtime systems maintain this to give easy access to (some) <i>static</i> or <i>extern</i> variables                                                            |
| 29     | <b>sp</b>     | Stack pointer                                                                                                                                                                        |
| 30     | <b>s8/fp</b>  | Ninth register variable; subroutines that need one can use this as a frame pointer                                                                                                   |
| 31     | <b>ra</b>     | Return address for subroutine                                                                                                                                                        |



# Usage of MIPS Registers

- \$a0 – \$a3: arguments (reg's 4 – 7)
- \$v0, \$v1: result values (reg's 2 and 3)
- \$t0 – \$t9: temporaries
  - Can be overwritten by callee
- \$s0 – \$s7: saved
  - Must be saved/restored by callee
- \$gp: global pointer for static data (reg 28)
- \$sp: stack pointer (reg 29)
- \$fp: frame pointer (reg 30)
- \$ra: return address (reg 31)

# C, Java Variables vs. Registers

- In C (and most HLLs):
  - Variables declared and given a type
    - Example:

```
int fahr, celsius;
char a, b, c, d, e;
```
  - Each variable can ONLY represent a value of the type it was declared
- In Assembly Language:
  - Registers have no type;
  - Operation determines how register contents are interpreted

# MIPS instruction types



Table 3-1 CPU Arithmetic Instructions

| Mnemonic | Instruction                                   |
|----------|-----------------------------------------------|
| ADD      | Add Word                                      |
| ADDI     | Add Immediate Word                            |
| ADDIU    | Add Immediate Unsigned Word                   |
| ADDU     | Add Unsigned Word                             |
| CLO      | Count Leading Ones in Word                    |
| CLZ      | Count Leading Zeros in Word                   |
| DIV      | Divide Word                                   |
| DIVU     | Divide Unsigned Word                          |
| MADD     | Multiply and Add Word to Hi, Lo               |
| MADDU    | Multiply and Add Unsigned Word to Hi, Lo      |
| MSUB     | Multiply and Subtract Word to Hi, Lo          |
| MSUBU    | Multiply and Subtract Unsigned Word to Hi, Lo |
| MUL      | Multiply Word to GPR                          |
| MULT     | Multiply Word                                 |
| MULTU    | Multiply Unsigned Word                        |
| SLT      | Set on Less Than                              |
| SLTI     | Set on Less Than Immediate                    |
| SLTIU    | Set on Less Than Immediate Unsigned           |
| SLTU     | Set on Less Than Unsigned                     |
| SUB      | Subtract Word                                 |
| SUBU     | Subtract Unsigned Word                        |

# MIPS instruction types



|    |                                                                                                                                                                                                      |    |    |    |    |    |    |    |            |   |               |   |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|----|----|----|----|------------|---|---------------|---|
| 编码 | 31                                                                                                                                                                                                   | 26 | 25 | 21 | 20 | 16 | 15 | 11 | 10         | 6 | 5             | 0 |
|    | special<br>000000                                                                                                                                                                                    |    | rs |    | rt |    | rd |    | 0<br>00000 |   | add<br>100000 |   |
|    | 6                                                                                                                                                                                                    |    | 5  |    | 5  |    | 5  |    | 5          |   | 6             |   |
| 格式 | add rd, rs, rt                                                                                                                                                                                       |    |    |    |    |    |    |    |            |   |               |   |
| 描述 | GPR[rd] ← GPR[rs]+GPR[rt]                                                                                                                                                                            |    |    |    |    |    |    |    |            |   |               |   |
| 操作 | <pre>temp ← (GPR[rs]<sub>31</sub>  GPR[rs]) + (GPR[rt]<sub>31</sub>  GPR[rt]) if temp<sub>32</sub> ≠ temp<sub>31</sub> then     SignalException(IntegerOverflow) else     GPR[rd] ← temp endif</pre> |    |    |    |    |    |    |    |            |   |               |   |
| 示例 | add \$s1, \$s2, \$s3                                                                                                                                                                                 |    |    |    |    |    |    |    |            |   |               |   |

|    |                                                                                                                                                                                                      |    |    |    |    |    |    |    |            |   |               |   |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|----|----|----|----|------------|---|---------------|---|
| 编码 | 31                                                                                                                                                                                                   | 26 | 25 | 21 | 20 | 16 | 15 | 11 | 10         | 6 | 5             | 0 |
|    | special<br>000000                                                                                                                                                                                    |    | rs |    | rt |    | rd |    | 0<br>00000 |   | sub<br>100010 |   |
|    | 6                                                                                                                                                                                                    |    | 5  |    | 5  |    | 5  |    | 5          |   | 6             |   |
| 格式 | sub rd, rs, rt                                                                                                                                                                                       |    |    |    |    |    |    |    |            |   |               |   |
| 描述 | GPR[rd] ← GPR[rs] - GPR[rt]                                                                                                                                                                          |    |    |    |    |    |    |    |            |   |               |   |
| 操作 | <pre>temp ← (GPR[rs]<sub>31</sub>  GPR[rs]) - (GPR[rt]<sub>31</sub>  GPR[rt]) if temp<sub>32</sub> ≠ temp<sub>31</sub> then     SignalException(IntegerOverflow) else     GPR[rd] ← temp endif</pre> |    |    |    |    |    |    |    |            |   |               |   |

# Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

add a, b, c # a gets b + c
- All arithmetic operations have this form
- *Design Principle 1: Simplicity favors regularity*
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

# Arithmetic Example

- C code:

```
f = (g + h) - (i + j);
```

- Compiled MIPS code:

```
add t0, g, h # temp t0 = g + h
add t1, i, j # temp t1 = i + j
sub f, t0, t1 # f = t0 - t1
```

- A single line of C may turn into several mips instructions

# Register Operands

- Arithmetic instructions use register operands
- MIPS has a  $32 \times 32$ -bit register file
  - Use for **frequently accessed** data
  - Numbered 0 to 31
  - 32-bit data called a “word”
- Assembler names
  - \$t0, \$t1, ..., \$t9 for temporary values
  - \$s0, \$s1, ..., \$s7 for saved variables
- **Design Principle 2: Smaller is faster**
  - c.f. main memory: millions of locations

|        |               |                                                                                                                                                                                      |
|--------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0      | <b>zero</b>   | Always returns 0                                                                                                                                                                     |
| 1      | <b>at</b>     | (assembly temporary) Reserved for use by assembly                                                                                                                                    |
| 2–3    | <b>v0, v1</b> | Value returned by subroutine                                                                                                                                                         |
| 4–7    | <b>a0–a3</b>  | (arguments) First few parameters for a subroutine                                                                                                                                    |
| 8–15   | <b>t0–t7</b>  | (temporaries) Subroutines can use without saving                                                                                                                                     |
| 24, 25 | <b>t8, t9</b> |                                                                                                                                                                                      |
| 16–23  | <b>s0–s7</b>  | Subroutine register variables; a subroutine that writes one of these must save the old value and restore it before it exits, so the <i>calling</i> routine sees the values preserved |
| 26, 27 | <b>k0, k1</b> | Reserved for use by interrupt/trap handler; may change under your feet                                                                                                               |
| 28     | <b>gp</b>     | Global pointer; some runtime systems maintain this to give easy access to (some) <i>static</i> or <i>extern</i> variables                                                            |
| 29     | <b>sp</b>     | Stack pointer                                                                                                                                                                        |
| 30     | <b>s8/fp</b>  | Ninth register variable; subroutines that need one can use this as a frame pointer                                                                                                   |
| 31     | <b>ra</b>     | Return address for subroutine <a href="http://blog.csdn.net/waixin_41560383">http://blog.csdn.net/waixin_41560383</a>                                                                |

# Register Operand Example

- C code:

`f = (g + h) - (i + j);`

- `f, ..., j` in `$s0, ..., $s4`

- Compiled MIPS code:

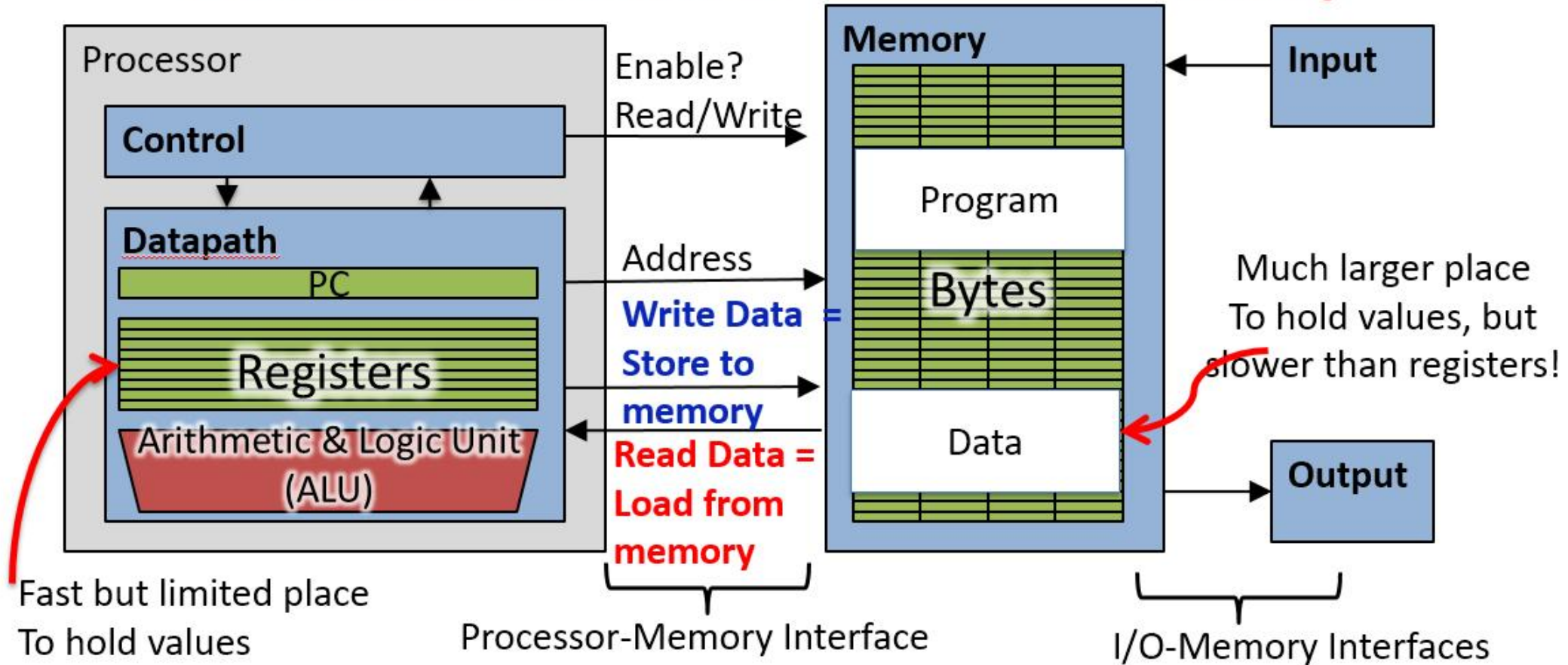
`add $t0, $s1, $s2`

`add $t1, $s3, $s4`

`sub $s0, $t0, $t1`



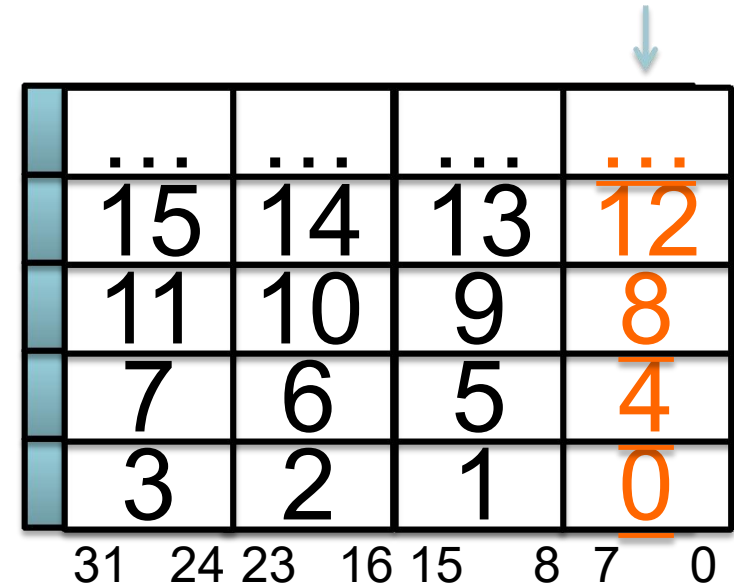
# Data Transfer: Load from and Store to memory



# Memory Addresses are in Bytes

- Data typically smaller than 32 bits, but rarely smaller than 8 bits (e.g., char type)—works fine if everything is a multiple of 8 bits
- 8 bit chunk is called a *byte*  
(1 word = 4 bytes)
- Memory addresses are really in *bytes*, not words
- Each address identifies an 8-bit byte

Least-significant byte in a word



|     |     |     |     |
|-----|-----|-----|-----|
| ... | ... | ... | ... |
| 15  | 14  | 13  | 12  |
| 11  | 10  | 9   | 8   |
| 7   | 6   | 5   | 4   |
| 3   | 2   | 1   | 0   |

31 24 23 16 15 8 7 0

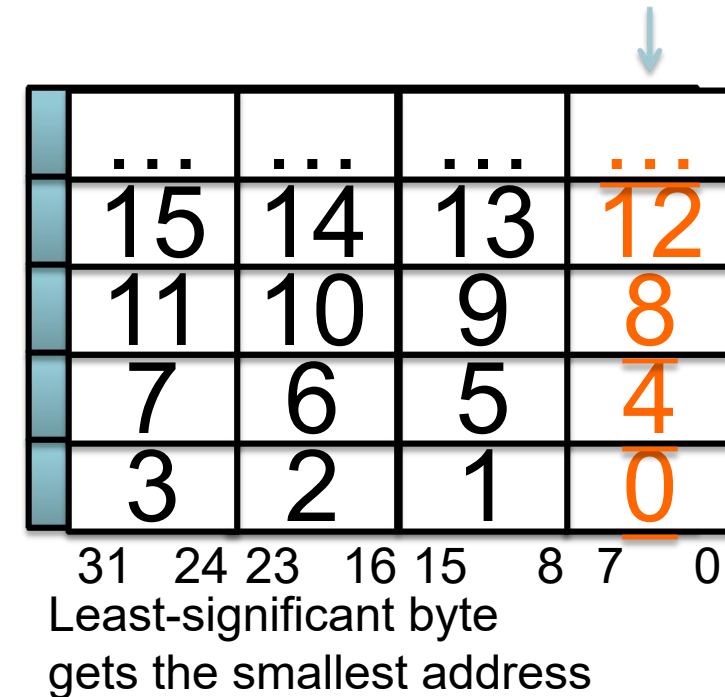
Least-significant byte

gets the smallest address

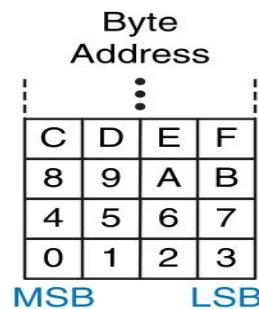
# Memory Addresses are in Bytes

- Words are **aligned in memory**(内存对齐)
  - Address must be a multiple of 4, the lowest two bits are 00
- Word addresses are 4 bytes apart
  - Word address is the smallest address
  - Little-endian convention
    - Least-significant byte at least address of a word
  - Big Endian convention
    - Most-significant byte at least address of a word
- MIPS is Big Endian

Least-significant byte in a word



Big-Endian



Little-Endian

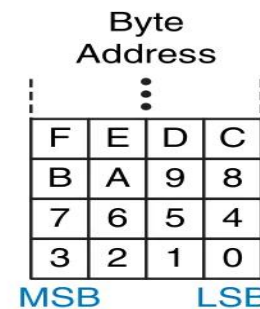


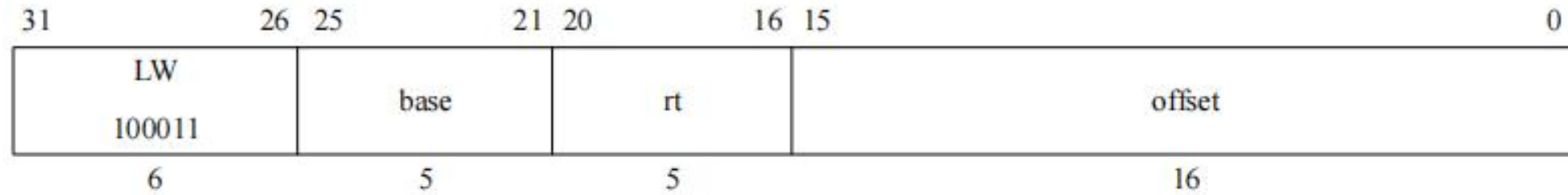
Figure 6.3 Big- and little-endian

# load/store

**Table 3-4 CPU Load, Store, and Memory Control Instructions**

| <b>Mnemonic</b> | <b>Instruction</b>     |
|-----------------|------------------------|
| LB              | Load Byte              |
| LBU             | Load Byte Unsigned     |
| LH              | Load Halfword          |
| LHU             | Load Halfword Unsigned |
| LL              | Load Linked Word       |
| LW              | Load Word              |
| LWL             | Load Word Left         |
| LWR             | Load Word Right        |
| PREF            | Prefetch               |
| SB              | Store Byte             |
| SC              | Store Conditional Word |
| SD              | Store Doubleword       |
| SH              | Store Halfword         |
| SW              | Store Word             |

# LW



**Format:** LW *rt*, *offset*(*base*)

**MIPS32**

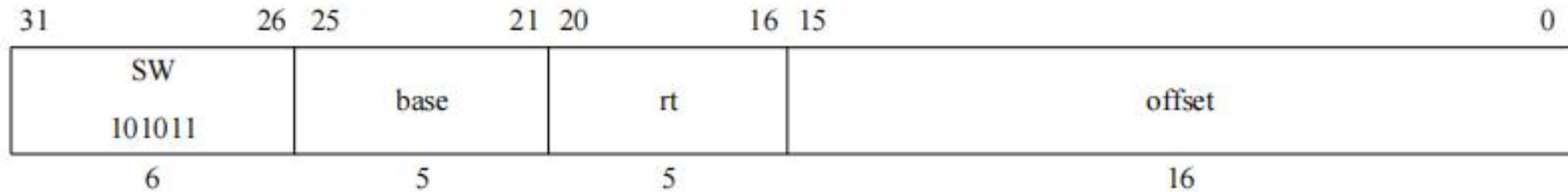
**Purpose:**

To load a word from memory as a signed value

**Description:**  $rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

The contents of the 32-bit word at the memory location specified by the aligned effective address are fetched, sign-extended to the GPR register length if necessary, and placed in GPR *rt*. The 16-bit signed *offset* is added to the contents of GPR *base* to form the effective address.

# SW



**Format:** SW *rt*, *offset*(*base*)

**MIPS32**

**Purpose:**

To store a word to memory

**Description:**  $\text{memory}[\text{base} + \text{offset}] \leftarrow \text{rt}$

The least-significant 32-bit word of register *rt* is stored in memory at the location specified by the aligned effective address. The 16-bit signed *offset* is added to the contents of GPR *base* to form the effective address.

# Memory Operand Example:

## Transfer from Memory to Register

- C code:

```
g = h + A[8];
```

- assume that g in \$s1, h in \$s2, base address of A in \$s3

- Compiled MIPS code:

- Index 8 requires offset of  $32=(8*4)$

- 4 bytes per word

```
lw $t0, 32($s3) # load word
add $s1, $s2, $t0
```

Offset in bytes

base register

Note: Offset must be a constant known at assembly time



# Memory Operand Example:

## Transfer from Register to Memory

- C code:

`A[12] = h + A[8];`

- `h` in `$s2`, base address of `A` in `$s3`

- Compiled MIPS code:

- Index 8 requires offset of 32

```
lw $t0, 32($s3) # load word
add $t0, $s2, $t0
sw $t0, 48($s3) # store word
```



# Loading and Storing **Byte** data

- In addition to word data transfers ( $lw$ ,  $sw$ ), MIPS has **byte** data transfers:
  - load byte:  $lb$
  - store byte:  $sb$
- Same format as  $lw$ ,  $sw$
- E.g.,  $lb \$s2, 3(\$s1)$ 
  - one byte in memory with address = sum of “3” + contents of register  $\$s1$  is copied to the low byte position of register  $\$s2$ .

$\$s2$      $xxxx$   $xxxx$   $xxxx$   $xxxx$   $xxxx$   $xxxx$   $xzzz$   $zzzz$

⬅ This bit is sign-extend

byte    loaded

# Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data requires **loads** and **stores**
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Register optimization is important!

# Immediate Operands

- Constant data specified in an instruction

`addi $s3, $s3, 4`

- No subtract immediate instruction

- Just use a negative constant

`addi $s2, $s1, -1`

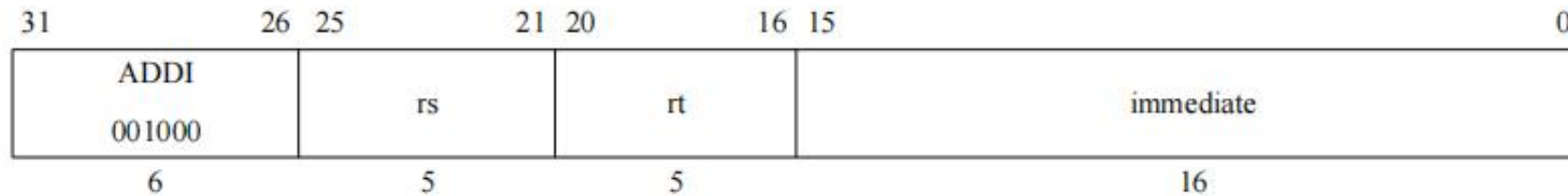
- *Design Principle 3: Make the common case fast*

- Small constants are common

- Immediate operand **avoids a load instruction**

## Add Immediate Word

**ADDI**



**Format:** `ADDI rt, rs, immediate`

**MIPS32**

### Purpose:

To add a constant to a 32-bit integer. If overflow occurs, then trap.

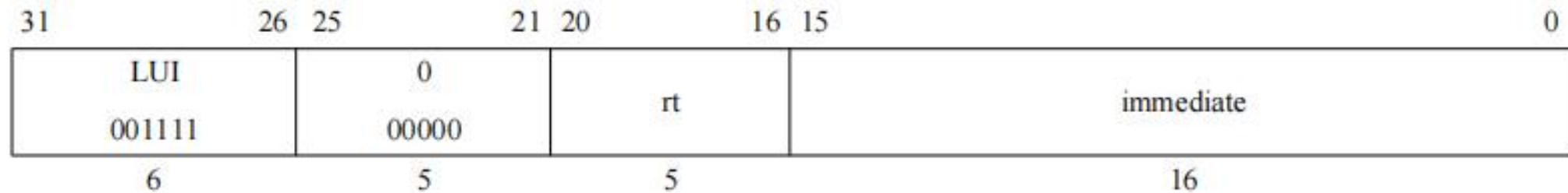
**Description:**  $rt \leftarrow rs + \text{immediate}$

The 16-bit signed *immediate* is added to the 32-bit value in GPR *rs* to produce a 32-bit result.

- If the addition results in 32-bit 2's complement arithmetic overflow, the destination register is not modified and an Integer Overflow exception occurs.
- If the addition does not overflow, the 32-bit result is placed into GPR *rt*.

## Load Upper Immediate

LUI



**Format:** LUI *rt*, *immediate*

**MIPS32**

### Purpose:

To load a constant into the upper half of a word

**Description:**  $rt \leftarrow \text{immediate} \parallel 0^{16}$

The 16-bit *immediate* is shifted left 16 bits and concatenated with 16 bits of low-order zeros. The 32-bit result is placed into GPR *rt*.

# The Constant Zero

- MIPS register 0 (\$zero) is the constant 0
  - Cannot be overwritten
- Useful for common operations
  - E.g., move between registers  
add \$t2, \$s1, \$zero

# Recap: Binary Number Conversion

## Binary → Decimal

$$1001010_{\text{two}} = ?_{\text{ten}}$$

| Binary Digit | Decimal Value              |
|--------------|----------------------------|
| 0            | $0 \times 2^0 = 0$         |
| 1            | $1 \times 2^1 = 2$         |
| 0            | $0 \times 2^2 = 0$         |
| 1            | $1 \times 2^3 = 8$         |
| 0            | $0 \times 2^4 = 0$         |
| 0            | $0 \times 2^5 = 0$         |
| 1            | $1 \times 2^6 = 64$        |
|              | $\Sigma = 74_{\text{ten}}$ |

## Decimal → Binary

$$74_{\text{ten}} = ?_{\text{two}}$$

| Decimal   | Binary (odd?)          |
|-----------|------------------------|
| 74        | 0                      |
| $/2 = 37$ | 1                      |
| $/2 = 18$ | 0                      |
| $/2 = 9$  | 1                      |
| $/2 = 4$  | 0                      |
| $/2 = 2$  | 0                      |
| $/2 = 1$  | 1                      |
| Collect → | $1001010_{\text{two}}$ |

# Signed Integer Representation

How to represent -5?

Sign & magnitude (8-bit example):

|      |                             |
|------|-----------------------------|
| sign | 7-bit magnitude (0 ... 127) |
|------|-----------------------------|

Rules for addition,  $a + b$ :

- If ( $a > 0$  and  $b > 0$ ): add  $b$  to  $a$
- If ( $a > 0$  and  $b < 0$ ): subtract  $b$  from  $a$
- ...
- $+0$ ,  $-0 \rightarrow$  are they equal? If so, comparator must handle special case! (If not equal???)

Cumbersome

- "Complicated" hardware: reduced speed / increased power
- ***Is there a better way?***



# Going backwards by going forwards

- How to add -1 to a four-bit number?

$$\begin{array}{rcl} 0111_{\text{two}} & = & 7_{\text{ten}} \\ \underline{????}_{\text{two}} + & = & -1_{\text{ten}} \\ 0110_{\text{two}} & = & 6_{\text{ten}} \end{array}$$

# Going backwards by going forwards

- Add  $10000_{\text{two}}$  and throw away fifth bit, result unchanged
  - Result wraps around to same value

$$\begin{array}{rcl} 0111_{\text{two}} & = & 7_{\text{ten}} \\ \underline{10000_{\text{two}}} + & = & 16_{\text{ten}} \\ 10111_{\text{two}} & = & 7_{\text{ten}} + 16_{\text{ten}} \\ 0111_{\text{two}} & = & 7_{\text{ten}} \end{array}$$

# Going backwards by going forwards

- Add ( $10000_{\text{two}} - 1_{\text{two}} = 01111_{\text{two}}$ ) and throw away fifth bit
  - One less than original value

$$\begin{array}{rcl} & 0111_{\text{two}} & = \\ \underline{01111}_{\text{two}} & + & = 16 - 1_{\text{ten}} \\ 10110_{\text{two}} & = & 16 + 6_{\text{ten}} \\ 0110_{\text{two}} & = & 6_{\text{ten}} \end{array}$$

# 2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range:  $-2^{n-1}$  to  $+2^{n-1} - 1$

- Example

- $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$   
 $= -1 \times 2^{31} + 1 \times 2^{30} + \dots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$   
 $= -2,147,483,648 + 2,147,483,644 = -4_{10}$

- Using 32 bits

- $-2,147,483,648$  to  $+2,147,483,647$

## ■ Two's Complement (8-bit example)

| As Signed<br>Decimal | As Unsigned<br>Decimal | Binary Number |
|----------------------|------------------------|---------------|
| -128                 | 128                    | 1000 0000     |
| -127                 | 129                    | 1000 0001     |
| ...                  | ...                    | ...           |
| -2                   | 254                    | 1111 1110     |
| -1                   | 255                    | 1111 1111     |
| 0                    | 0                      | 0000 0000     |
| 1                    | 1                      | 0000 0001     |
| ...                  | ...                    | ...           |
| 127                  | 127                    | 0111 1111     |

Most-significant bit (MSB) equals sign when interpreted as a signed number

# Signed versus Unsigned Binary Numbers

- Both are stored as a collection of bits
- The same set of bits can be used to represent a **signed** number **or** an **unsigned** number (or a string, or a picture, or a....)
- It's up to the **operation** interpreting the bits

$1101_{\text{two}}$  =  $+13_{\text{ten}}$  (as an unsigned number)

$1101_{\text{two}}$  =  $-3_{\text{ten}}$  (as a signed number)

$1101_{\text{two}}$  = orange (as a color)

$1101_{\text{two}}$  = cat (as a type of animal)

$1101_{\text{two}}$  = *whatever you want it to mean...*

# Unary Negation (Two's Complement)

## 4-bit Example ( $-8_{\text{ten}} \dots +7_{\text{ten}}$ )

### Brute Force & Tedious

16<sub>ten</sub>  
- 3<sub>ten</sub>  
-----  
13<sub>ten</sub>

16<sub>ten</sub>  
- 13<sub>ten</sub>  
-----  
3<sub>ten</sub>

10000<sub>two</sub>  
- 0011<sub>two</sub>  
-----  
1101<sub>two</sub>

10000<sub>two</sub>  
- 1101<sub>two</sub>  
-----  
0011<sub>two</sub>

"largest" 4-bit number + 1

### Clever & Elegant

15<sub>ten</sub>  
- 3<sub>ten</sub>  
-----  
12<sub>ten</sub>  
+ 1<sub>ten</sub>  
-----  
13<sub>ten</sub>

01111<sub>two</sub>  
- 0011<sub>two</sub>  
-----  
1100<sub>two</sub>  
+ 0001<sub>two</sub>  
-----  
1101<sub>two</sub>

*invert*

# Your Turn

- What is the decimal value of the following binary 8-bit 2's complement number?

1110 0001<sub>two</sub>

| Answer | Value              |
|--------|--------------------|
| RED    | 33 <sub>ten</sub>  |
| ORANGE | -31 <sub>ten</sub> |
| GREEN  | 225 <sub>ten</sub> |
| YELLOW | -33 <sub>ten</sub> |



# Addition

## 4-bit Example

### Unsigned

$$\begin{array}{r} 3_{\text{ten}} \\ + 4_{\text{ten}} \\ \hline 7_{\text{ten}} \end{array} \quad \begin{array}{r} 0011_{\text{two}} \\ + 0100_{\text{two}} \\ \hline 0111_{\text{two}} \end{array}$$

$$\begin{array}{r} 3_{\text{ten}} \\ + 11_{\text{ten}} \\ \hline 14_{\text{ten}} \end{array} \quad \begin{array}{r} 0011_{\text{two}} \\ + 1011_{\text{two}} \\ \hline 1110_{\text{two}} \end{array}$$

### Signed (Two's Complement)

$$\begin{array}{r} 3_{\text{ten}} \\ + 4_{\text{ten}} \\ \hline 7_{\text{ten}} \end{array} \quad \begin{array}{r} 0011_{\text{two}} \\ + 0100_{\text{two}} \\ \hline 0111_{\text{two}} \end{array}$$

$$\begin{array}{r} 3_{\text{ten}} \\ + -5_{\text{ten}} \\ \hline -2_{\text{ten}} \end{array} \quad \begin{array}{r} 0011_{\text{two}} \\ + 1011_{\text{two}} \\ \hline 1110_{\text{two}} \end{array}$$

*No special rules for two's complement signed addition*

# Overflow

## 4-bit Example

### Unsigned addition

$$\begin{array}{r} 13_{\text{ten}} \\ + 14_{\text{ten}} \\ \hline 27_{\text{ten}} \end{array} \quad \begin{array}{r} 1101_{\text{two}} \\ + 1110_{\text{two}} \\ \hline \textcircled{1}1011_{\text{two}} \end{array}$$

carry-out and overflow

$$\begin{array}{r} 7_{\text{ten}} \\ + 1_{\text{ten}} \\ \hline 8_{\text{ten}} \end{array} \quad \begin{array}{r} 0111_{\text{two}} \\ + 0001_{\text{two}} \\ \hline \textcircled{0}1000_{\text{two}} \end{array}$$

no carry-out and no overflow

*Carry-out → Overflow*

### Signed addition (Two's Complement)

$$\begin{array}{r} -3_{\text{ten}} \\ + -2_{\text{ten}} \\ \hline -5_{\text{ten}} \end{array} \quad \begin{array}{r} 1101_{\text{two}} \\ + 1110_{\text{two}} \\ \hline \textcircled{1}1011_{\text{two}} \end{array}$$

carry-out but no overflow

$$\begin{array}{r} 7_{\text{ten}} \\ + 1_{\text{ten}} \\ \hline -8_{\text{ten}} \end{array} \quad \begin{array}{r} 0111_{\text{two}} \\ + 0001_{\text{two}} \\ \hline \textcircled{0}1000_{\text{two}} \end{array}$$

no carry-out but overflow

*Carry-out → ~~Overflow~~*

# Addition Overflow Detection

## 4-bit Example

### Unsigned addition

- Carry-out indicates overflow

### Signed addition (Two's Complement)

- Overflow if
  - Signs of operands are equal  
*AND*
  - Sign of result differs from sign of operands
- No overflow is possible when signs of operands differ

***Overflow rules depend on the operation (signed vs unsigned addition)***

# Adding Numbers of Different Bit Widths

Unsigned addition

$$\begin{array}{r} 1101 \quad (4\text{-bit}, 13_{\text{ten}}) \\ 10100101 \quad (8\text{-bit}, 165_{\text{ten}}) \\ \hline ???0010 \quad (8\text{-bit}, ???_{\text{ten}}) \end{array}$$

Zero extension

$$\begin{array}{r} 00001101 \quad (4\text{-bit}, 13_{\text{ten}}) \\ 10100101 \quad (8\text{-bit}, 165_{\text{ten}}) \\ \hline 10110010 \quad (8\text{-bit}, 178_{\text{ten}}) \end{array}$$

- What should we assume for upper four bits of first operand?
- Treat as zeros!
- “Zero-extension”

# Adding Signed Numbers of Different Bit Widths

Sign bit

Signed addition

$\begin{array}{r} \textcolor{blue}{1}101 \quad (4\text{-bit, } -3_{\text{ten}}) \\ 10100101 \quad (8\text{-bit, } -91_{\text{ten}}) \\ \hline \text{????}0010 \quad (8\text{-bit, } ???_{\text{ten}}) \end{array}$

Sign extension

$\begin{array}{r} \textcolor{red}{1111}\textcolor{blue}{1}101 \quad (4\text{-bit, } -3_{\text{ten}}) \\ 10100101 \quad (8\text{-bit, } -91_{\text{ten}}) \\ \hline 10100010 \quad (8\text{-bit, } -94_{\text{ten}}) \end{array}$

- What should we assume for upper four bits of first operand?
- **Copy sign bit** (MSB) of operand into upper bits
  - If sign is 1, copy 1s, if sign is 0 copy 0s
- “Sign-extension”

# Sign Extension

| Decimal           | Binary              |                          |                                    |
|-------------------|---------------------|--------------------------|------------------------------------|
|                   | 4-bit               | 8-bit                    | 32-bit                             |
| 3 <sub>ten</sub>  | 0011 <sub>two</sub> | 0000 0011 <sub>two</sub> | 0000 0000 0000 0011 <sub>two</sub> |
| -3 <sub>ten</sub> | 1101 <sub>two</sub> | 1111 1101 <sub>two</sub> | 1111 1111 1111 1101 <sub>two</sub> |

- When is this relevant?
- Any time you need a wider version of a narrower operand  
e.g., adding two integers of different widths

# Your Turn

- Which range of decimals can be expressed with a 6-bit two's complement number?

| Answer | Range      |
|--------|------------|
| RED    | -32 ... 32 |
| GREEN  | -64 ... 63 |
| ORANGE | -31 ... 32 |
| YELLOW | -32 ... 31 |

# Recap: 2s-Complement Signed Integers

- Bit 31 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- $-(-2^n - 1)$  can't be represented
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
  - 0: 0000 0000 ... 0000
  - -1: 1111 1111 ... 1111
  - Most-negative: 1000 0000 ... 0000
  - Most-positive: 0111 1111 ... 1111



# Recap: Signed Negation

- Complement and add 1
  - Complement means  $1 \rightarrow 0, 0 \rightarrow 1$

$$x + \bar{x} = 1111 \dots 111_2 = -1$$

$$\bar{x} + 1 = -x$$

- Example: negate +2
  - $+2 = 0000 \ 0000 \dots 0010_2$
  - $-2 = 1111 \ 1111 \dots 1101_2 + 1$   
 $= 1111 \ 1111 \dots 1110_2$

# Recap: Sign Extension

- Representing a number using more bits
  - Preserve the numeric value
- In MIPS instruction set
  - `addi`: extend immediate value
  - `lb`, `lh`: extend loaded byte/halfword
  - `beq`, `bne`: extend the displacement
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - -2: 1111 1110 => 1111 1111 1111 1110

# Representing Instructions

- Instructions are encoded in binary
  - Called machine code
- MIPS instructions
  - Encoded as 32-bit instruction words
  - Small number of formats encoding operation code (opcode), register numbers, ...
  - Regularity!
- Register numbers
  - \$t0 – \$t7 are reg's 8 – 15
  - \$t8 – \$t9 are reg's 24 – 25
  - \$s0 – \$s7 are reg's 16 – 23

# Hexadecimal

- Base 16
  - Compact representation of bit strings
  - 4 bits per hex digit

|   |      |   |      |   |      |   |      |
|---|------|---|------|---|------|---|------|
| 0 | 0000 | 4 | 0100 | 8 | 1000 | c | 1100 |
| 1 | 0001 | 5 | 0101 | 9 | 1001 | d | 1101 |
| 2 | 0010 | 6 | 0110 | a | 1010 | e | 1110 |
| 3 | 0011 | 7 | 0111 | b | 1011 | f | 1111 |

- Example: eca8 6420
  - 1110 1100 1010 1000 0110 0100 0010 0000

# MIPS R-format Instructions



## ■ Instruction fields

- op: operation code (opcode)
- rs: first source register number
- rt: second source register number
- rd: destination register number
- shamt: shift amount (00000 for now)
- funct: function code (extends opcode)

# R-format Example

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| op     | rs     | rt     | rd     | shamt  | funct  |
| 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |

add \$t0, \$s1, \$s2

|         |       |       |       |       |        |
|---------|-------|-------|-------|-------|--------|
| special | \$s1  | \$s2  | \$t0  | 0     | add    |
| 0       | 17    | 18    | 8     | 0     | 32     |
| 000000  | 10001 | 10010 | 01000 | 00000 | 100000 |

$00000010001100100100000000100000_2 = 02324020_{16}$

# MIPS I-format Instructions



- Immediate arithmetic and load/store instructions
  - rt: destination or source register number
  - Constant:  $-2^{15}$  to  $+2^{15} - 1$
  - Address: offset added to base address in rs
- *Design Principle 4: Good design demands good compromises*
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

# Logical Operations

- Instructions for bitwise manipulation

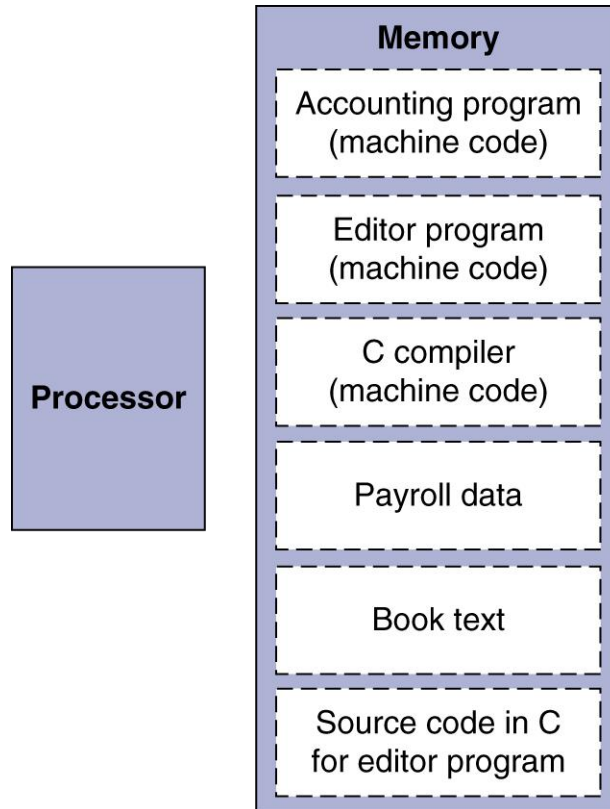
| Operation   | C  | Java | MIPS      |
|-------------|----|------|-----------|
| Shift left  | << | <<   | sll       |
| Shift right | >> | >>>  | srl       |
| Bitwise AND | &  | &    | and, andi |
| Bitwise OR  |    |      | or, ori   |
| Bitwise NOT | ~  | ~    | nor       |

- Useful for extracting and inserting groups of bits in a word



# Stored Program Computers

## The BIG Picture



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, ...
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

# Shift Operations

|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| op     | rs     | rt     | rd     | shamt  | funct  |
| 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |

- shamt: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - srl by  $i$  bits multiplies by  $2^i$
- Shift right logical
  - Shift right and fill with 0 bits
  - srl by  $i$  bits divides by  $2^i$  (unsigned only)

# AND Operations

- Useful to mask bits in a word
  - Select some bits, clear others to 0

and \$t0, \$t1, \$t2

|      |                                         |
|------|-----------------------------------------|
| \$t2 | 0000 0000 0000 0000 0000 1101 1100 0000 |
| \$t1 | 0000 0000 0000 0000 0011 1100 0000 0000 |
| \$t0 | 0000 0000 0000 0000 0000 1100 0000 0000 |

# OR Operations

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged

or \$t0, \$t1, \$t2

|      |                                         |
|------|-----------------------------------------|
| \$t2 | 0000 0000 0000 0000 0000 1101 1100 0000 |
| \$t1 | 0000 0000 0000 0000 0011 1100 0000 0000 |
| \$t0 | 0000 0000 0000 0000 0011 1101 1100 0000 |

# NOT Operations

- Useful to invert bits in a word
  - Change 0 to 1, and 1 to 0
- MIPS has NOR 3-operand instruction
  - $a \text{ NOR } b == \text{NOT} (a \text{ OR } b)$

`nor $t0, $t1, $zero` ←

Register 0: always  
read as zero

\$t1    0000 0000 0000 0000 0011 1100 0000 0000

\$t0    1111 1111 1111 1111 1100 0011 1111 1111

# Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- `beq rs, rt, L1`
  - if (`rs == rt`) branch to instruction labeled L1;
- `bne rs, rt, L1`
  - if (`rs != rt`) branch to instruction labeled L1;
- `j L1`
  - unconditional jump to instruction labeled L1

# Compiling If Statements

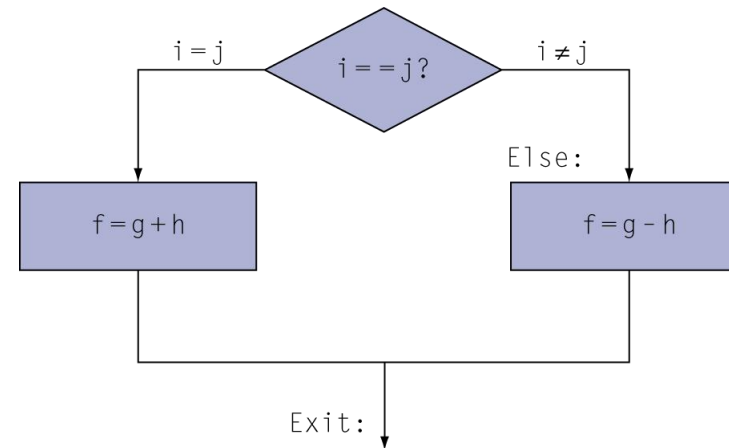
- C code:

```
if (i==j) f = g+h;
else f = g-h;
```

- f, g, ... in \$s0, \$s1, ...

- **Compiled MIPS code:**

```
 bne $s3, $s4, Else
 add $s0, $s1, $s2
 j Exit
Else: sub $s0, $s1, $s2
Exit: ...
```



Assembler calculates addresses

# Compiling Loop Statements

- C code:

```
while (save[i] == k) i += 1;
```

- i in \$s3, k in \$s5, address of save in \$s6

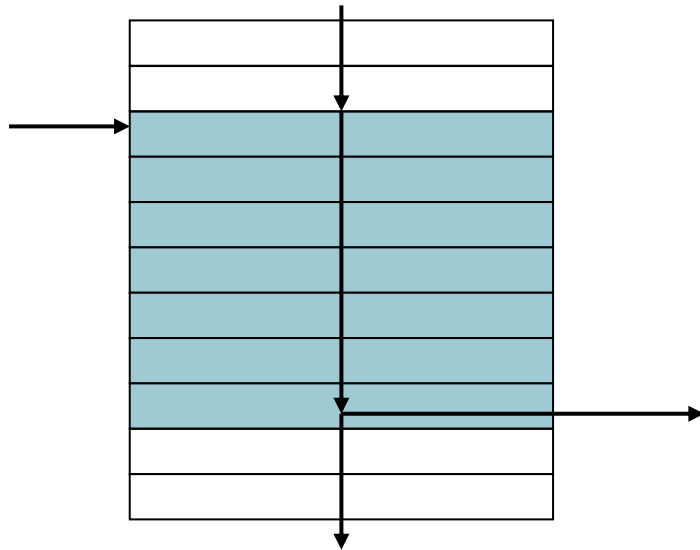
- Compiled MIPS code:

```
Loop: sll $t1, $s3, 2
 add $t1, $t1, $s6
 lw $t0, 0($t1)
 bne $t0, $s5, Exit
 addi $s3, $s3, 1
 j Loop
Exit: ...
```



# Basic Blocks

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)



- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks

# More Conditional Operations

- Set result to **1** if a condition is true

- Otherwise, set to 0

- **slt rd, rs, rt**

- if ( $rs < rt$ )  $rd = 1$ ; else  $rd = 0$ ;

- **slti rt, rs, constant**

- if ( $rs < \text{constant}$ )  $rt = 1$ ; else  $rt = 0$ ;

- Use in combination with beq, bne

```
slt $t0, $s1, $s2 # if ($s1 < $s2)
bne $t0, $zero, L # branch to L
```

# Branch Instruction Design

- Why not b1t, bge, etc?
- Hardware for  $<$ ,  $\geq$ , ... slower than  $=$ ,  $\neq$ 
  - Combining with branch involves more work per instruction, requiring a slower clock
  - All instructions penalized!
- beq and bne are the common case
- This is a good design compromise

# Signed vs. Unsigned

- Signed comparison: `slt`, `slti`
- Unsigned comparison: `sltu`, `sltui`
- Example
  - `$s0 = 1111 1111 1111 1111 1111 1111 1111 1111`
  - `$s1 = 0000 0000 0000 0000 0000 0000 0000 0001`
  - `slt $t0, $s0, $s1` # signed
    - $-1 < +1 \Rightarrow \$t0 = 1$
  - `sltu $t0, $s0, $s1` # unsigned
    - $+4,294,967,295 > +1 \Rightarrow \$t0 = 0$

# Procedure Calling

- Steps required
  1. (Caller) Place parameters in registers
  2. (Caller) Transfer control to procedure (Callee)
  3. (Caller) Acquire storage for procedure (Callee)
  4. (Callee) Perform procedure's operations
  5. (Callee) Place result in register for caller
  6. Return to place of call

# Register Usage

- \$v0, \$v1: result values (reg's 2 and 3)
- \$a0 – \$a3: arguments (reg's 4 – 7)
- \$t0 – \$t9: temporaries
  - Can be overwritten by callee
- \$s0 – \$s7: saved
  - Must be saved/restored by callee
- \$gp: global pointer for static data (reg 28)
- \$sp: stack pointer (reg 29)
- \$fp: frame pointer (reg 30)
- \$ra: return address (reg 31)

# Procedure Call Instructions

- Procedure call: **jal** (jump and link)

**jal** ProcedureLabel

- Puts address of following instruction of Caller in \$ra
- Jumps to Callee

- Procedure return: jump register

**jr** \$ra

- Copies \$ra to program counter
- Can also be used for computed jumps
  - e.g., for case/switch statements

# Leaf Procedure Example

- C code:

```
int leaf_example (int g, h, i, j)
{ int f;
 f = (g + h) - (i + j);
 return f;
}
```

- Arguments g, ..., j in \$a0, ..., \$a3
- f in \$s0 (hence, need to save \$s0 on stack)
- Result in \$v0



# Leaf Procedure Example

## ■ MIPS code:

leaf\_example:

```
addi $sp, $sp, -4
sw $s0, 0($sp)
```

```
add $t0, $a0, $a1
add $t1, $a2, $a3
sub $s0, $t0, $t1
```

```
add $v0, $s0, $zero
```

```
lw $s0, 0($sp)
addi $sp, $sp, 4
```

```
jr $ra
```

Save \$s0 on stack

Procedure body

Stack Pointer

Result

Restore \$s0

Return

```
int leaf_example (int g, h, i, j)
{ int f;
 f = (g + h) - (i + j);
 return f;
}
```

- Arguments g, ..., j in \$a0, ..., \$a3
- f in \$s0 (hence, need to save \$s0 on stack)
- Result in \$v0

Frame main

Frame leaf\_example  
int f (\$s0) ;

# Non-Leaf Procedures

- Procedures that call other procedures
- For **nested** call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call

# Non-Leaf Procedure Example

- C code:

```
int fact (int n)
{
 if (n < 1) return 1;
 else return n * fact(n - 1);
}
```

- Argument n in \$a0
- Result in \$v0

# Non-Leaf Procedure Example

```
int fact (int n)
{
 if (n < 1) return 1;
 else return n * fact(n - 1);
}
```

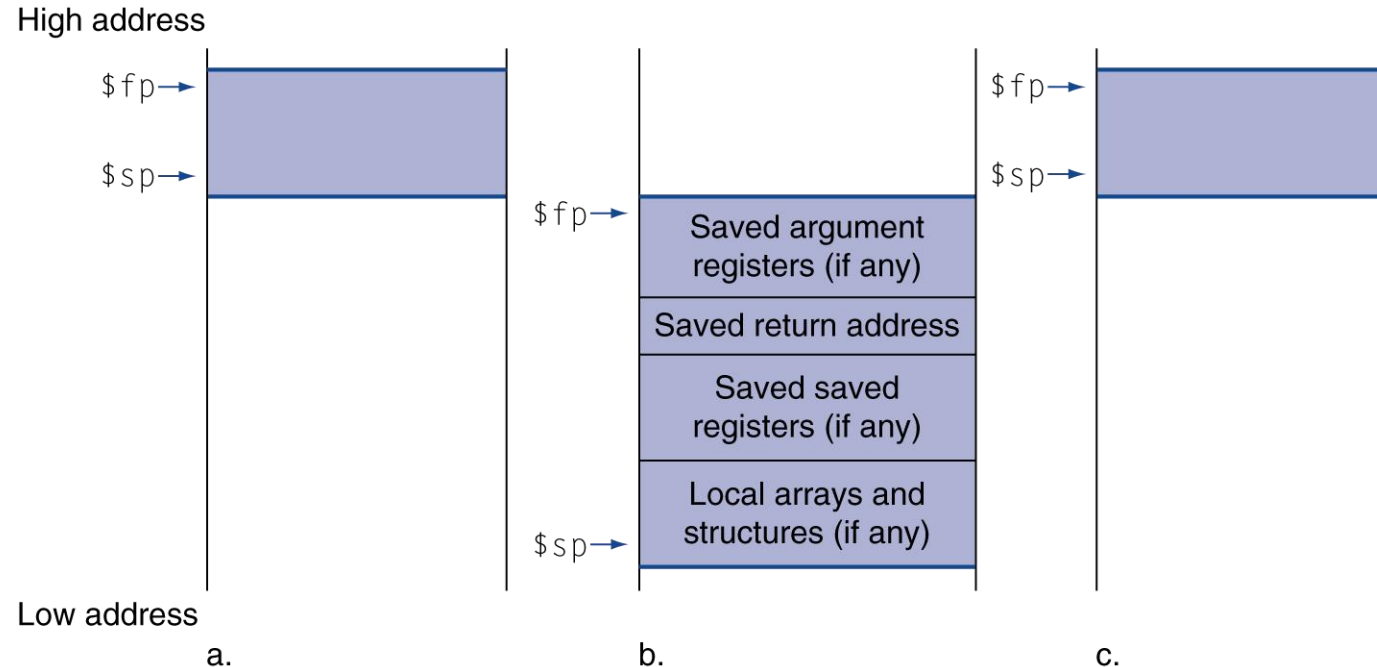
- Argument n in \$a0
- Result in \$v0

■ MIPS code:

|       |                  |                                       |
|-------|------------------|---------------------------------------|
| fact: |                  |                                       |
| addi  | \$sp, \$sp, -8   | # adjust stack for 2 items            |
| sw    | \$ra, 4(\$sp)    | # save return address                 |
| sw    | \$a0, 0(\$sp)    | # save argument (int n)               |
| slti  | \$t0, \$a0, 1    | # test for n < 1 if(n<1)\$t0=1 else 0 |
| beq   | \$t0, \$zero, L1 | # the case of n>1,branch to L1        |
| addi  | \$v0, \$zero, 1  | # if so, result is 1                  |
| addi  | \$sp, \$sp, 8    | # pop 2 items from stack              |
| jr    | \$ra             | # and return                          |
| L1:   |                  |                                       |
| addi  | \$a0, \$a0, -1   | # else decrement n                    |
| jal   | fact             | # recursive call                      |
| lw    | \$a0, 0(\$sp)    | # restore original n                  |
| lw    | \$ra, 4(\$sp)    | # and return address                  |
| addi  | \$sp, \$sp, 8    | # pop 2 items from stack              |
| mul   | \$v0, \$a0, \$v0 | # multiply to get result              |
| jr    | \$ra             | # and return                          |



# Local Data on the Stack

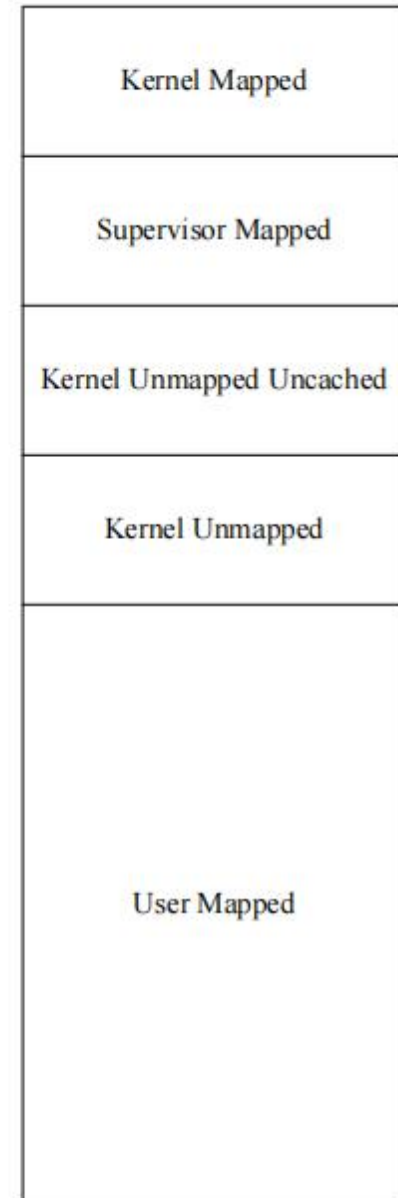


- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

# Memory Layout

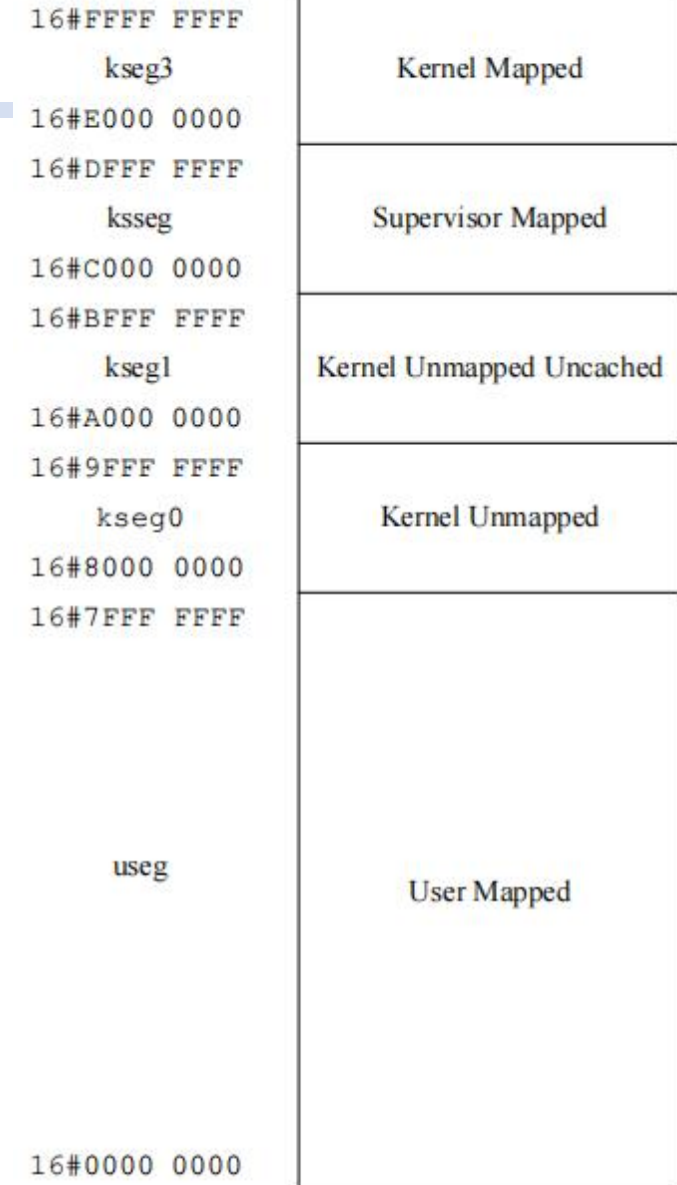
- **kuseg**: 0x000 0000 - 0x7FFF FFFF (低端2G): 用户模式下可用的地址，MIPS规范约定用户空间为2GB。
- **kseg0**: 0x8000 0000 - 0x9FFF FFFF(512MB)
- 只需把最高位清零（&0x7fffffff），这些地址就被转换为物理地址，然后连续映射到物理内存的低端512M(0x0000 0000 - 0x1FFF FFFF)空间。因为这种映射不需要MMU转换(Unmapped)。

16#FFFF FFFF  
kseg3  
16#E000 0000  
16#DFFF FFFF  
ksseg  
16#C000 0000  
16#BFFF FFFF  
kseg1  
16#A000 0000  
16#9FFF FFFF  
kseg0  
16#8000 0000  
16#7FFF FFFF  
  
useg  
  
16#0000 0000



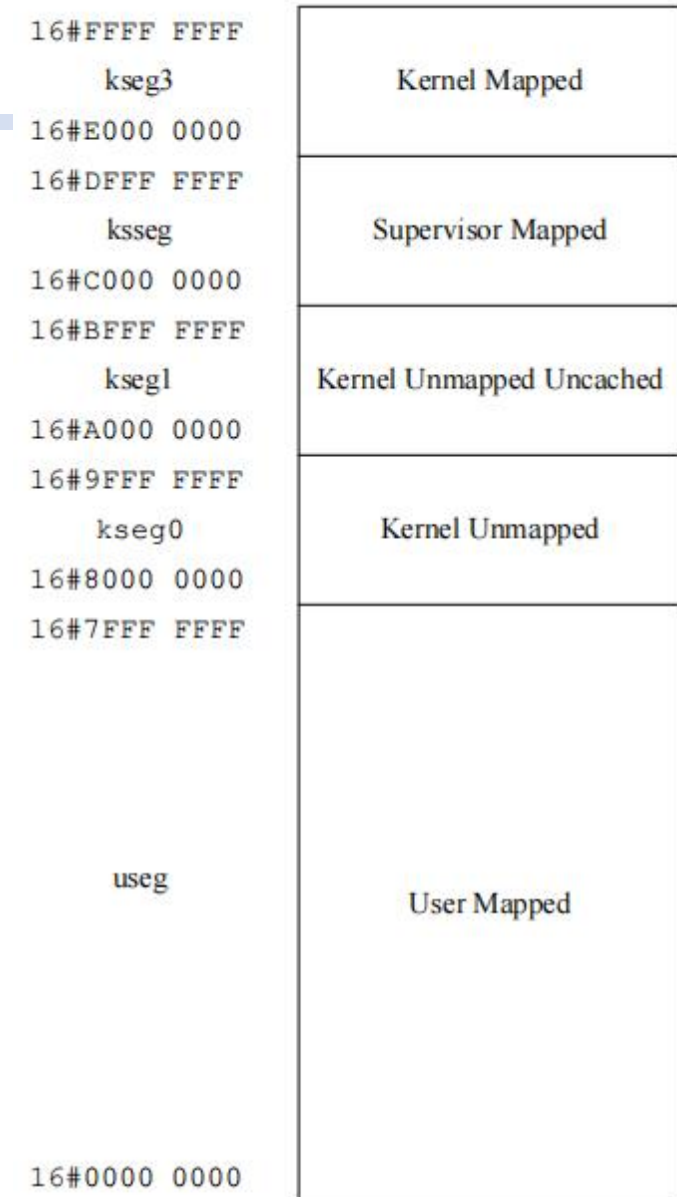
# Memory Layout

- **kseg1**: 0xA000 0000 - 0xBFFF FFFF(512M)
- 通过将最高3位清零（&0x1ffffff）的方法来把这些地址映射为相应的物理地址，与kseg0 映射的物理地址一样，都映射到物理内存的低端512M(0x0000 0000 - 0x1FFF FFFF)空间，也是“非翻译无需转换的”（Unmapped）地址区域。但要注意，kseg1不使用缓存（Uncached）。
- kseg1是唯一的在系统重启时能正常工作的内存映射地址空间，这也是为什么重新启动时的入口向量是（0xBFC0 0000）会在这个区域。这个向量对应的物理地址是0x1FC0 0000。



# Memory Layout

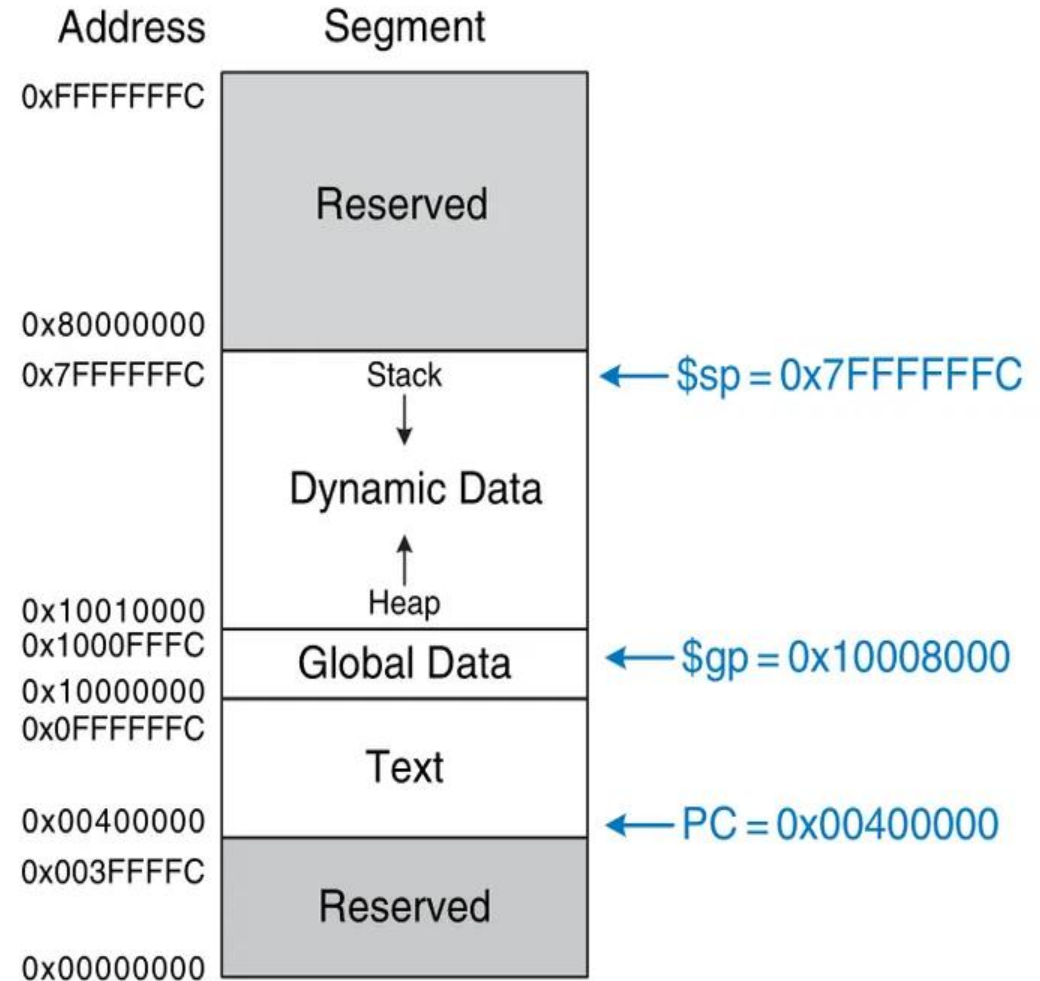
- **kseg/kseg3: 0xC000 0000 - 0xFFFF FFFF (1G)**
- 这段地址空间只能在核心态下使用并且要经过MMU转换。在MMU 设置好之前，不能存取这段区域。除非你在写一个真正的操作系统，一般来说你不需要使用这段地址空间。
- **kseg2**对于运行在监管者模式可用。
- 由于**kseg0**和**kseg1**用于操作系统分配外设I/O地址，加上**kseg/kseg3**的1G空间，故MIPS规范约定内核空间为2G。





# Memory Layout

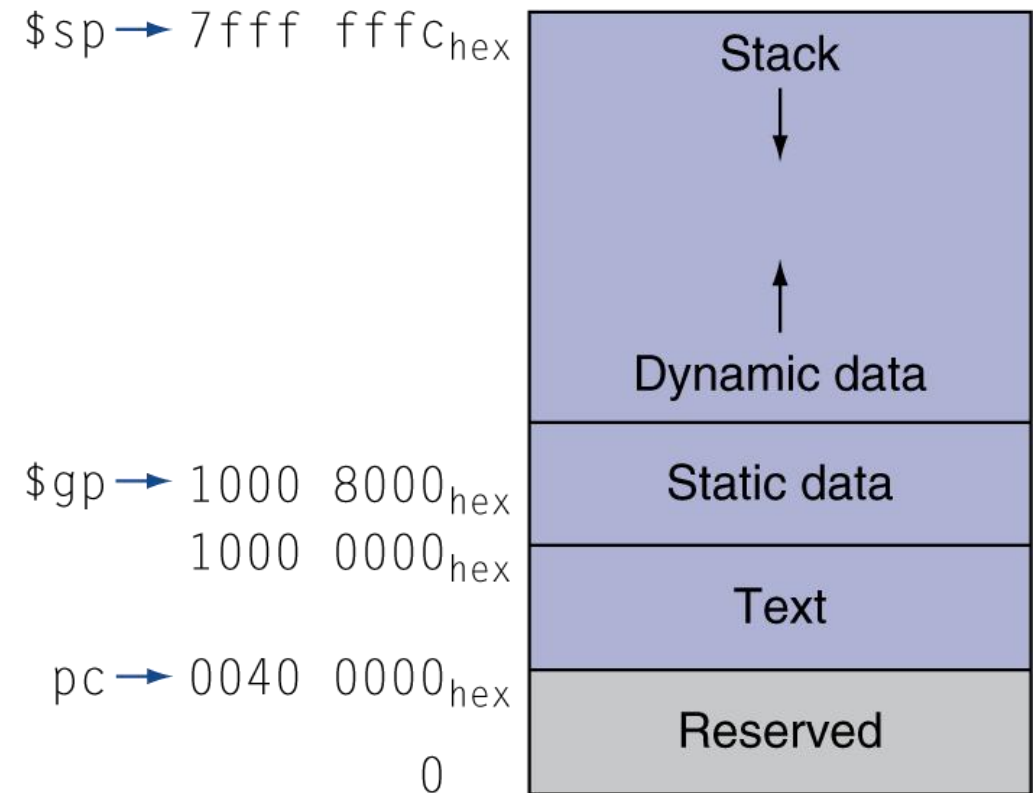
- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - \$gp initialized to address allowing  $\pm$ offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



**Figure 6.31** MIPS memory map

# Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - \$gp initialized to address allowing  $\pm$ offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



# Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, ...
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

# Byte/Halfword Operations

- Could use bitwise operations
- MIPS byte/halfword load/store
  - String processing is a common case

`lb rt, offset(rs)`      `lh rt, offset(rs)`

- Sign extend to 32 bits in rt

`lbu rt, offset(rs)`      `lhu rt, offset(rs)`

- Zero extend to 32 bits in rt

`sb rt, offset(rs)`      `sh rt, offset(rs)`

- Store just rightmost byte/halfword

# String Copy Example

- C code (naïve):

- Null-terminated string

```
void strcpy (char x[], char y[])
{ int i;
 i = 0;
 while ((x[i]=y[i])!='\0')
 i += 1;
}
```

- Addresses of x, y in \$a0, \$a1
- i in \$s0

# String Copy Example

## ■ MIPS code:

|         |                                           |
|---------|-------------------------------------------|
| strcpy: |                                           |
| addi    | \$sp, \$sp, -4 # adjust stack for 1 item  |
| sw      | \$s0, 0(\$sp) # save \$s0                 |
| add     | \$s0, \$zero, \$zero # i = 0              |
| L1: add | \$t1, \$s0, \$a1 # addr of y[i] in \$t1   |
| lbu     | \$t2, 0(\$t1) # \$t2 = y[i]               |
| add     | \$t3, \$s0, \$a0 # addr of x[i] in \$t3   |
| sb      | \$t2, 0(\$t3) # x[i] = y[i]               |
| beq     | \$t2, \$zero, L2 # exit loop if y[i] == 0 |
| addi    | \$s0, \$s0, 1 # i = i + 1                 |
| j       | L1 # next iteration of loop               |
| L2: lw  | \$s0, 0(\$sp) # restore saved \$s0        |
| addi    | \$sp, \$sp, 4 # pop 1 item from stack     |
| jr      | \$ra # and return                         |

# 32-bit Constants

- Most constants are small
  - 16-bit immediate is sufficient
- For the occasional 32-bit constant

**`lui rt, constant`**

- Copies 16-bit constant to left 16 bits of rt
- Clears right 16 bits of rt to 0

`lui $s0, 61`

|                     |                     |
|---------------------|---------------------|
| 0000 0000 0111 1101 | 0000 0000 0000 0000 |
|---------------------|---------------------|

`ori $s0, $s0, 2304`

|                     |                     |
|---------------------|---------------------|
| 0000 0000 0111 1101 | 0000 1001 0000 0000 |
|---------------------|---------------------|

# Branch Addressing

- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward

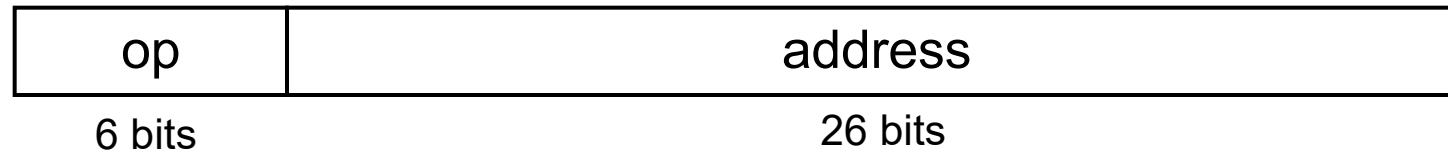


- PC-relative addressing
  - Target address =  $PC + \text{offset} \times 4$
  - PC already incremented by 4 by this time



# Jump Addressing

- Jump (j and jal) targets could be anywhere in text segment
  - Encode full address in instruction



- (Pseudo)Direct jump addressing
  - Target address =  $PC_{31..28} : (\text{address} \times 4)$

$$PC \leftarrow PC_{\text{GPRLEN}-1..28} \parallel \text{instr\_index} \parallel 0^2$$

# Target Addressing Example

- Loop code from earlier example
  - Assume Loop at location 80000

```
Loop: sll $t1, $s3, 2 80000
 add $t1, $t1, $s6 80004
 lw $t0, 0($t1) 80008
 bne $t0, $s5, Exit 80012
 addi $s3, $s3, 1 80016
 j Loop 80020
Exit: ... 80024
```

|    |       |    |   |   |    |
|----|-------|----|---|---|----|
| 0  | 0     | 19 | 9 | 4 | 0  |
| 0  | 9     | 22 | 9 | 0 | 32 |
| 35 | 9     | 8  | 0 |   |    |
| 5  | 8     | 21 | 2 |   |    |
| 8  | 19    | 19 | 1 |   |    |
| 2  | 20000 |    |   |   |    |
|    |       |    |   |   |    |

# Branching Far Away

- If branch target is too far to encode with 16-bit offset, assembler rewrites the code
- Example

beq \$s0,\$s1, L1 (L1 cannot be represented by 16 bits)



bne \$s0,\$s1, L2

j L1

L2: ...

# Addressing Mode Summary

## 1. Immediate addressing



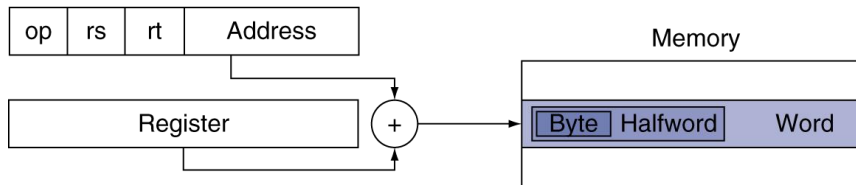
`addi $s3, $s3, 1`

## 2. Register addressing



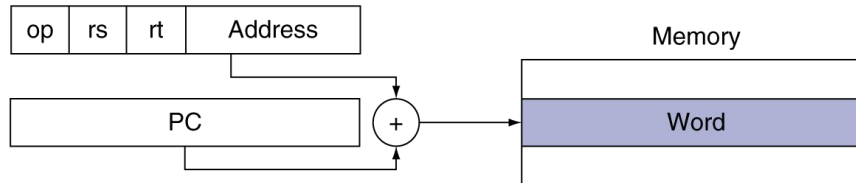
`add $t1, $t1, $s6`

## 3. Base addressing



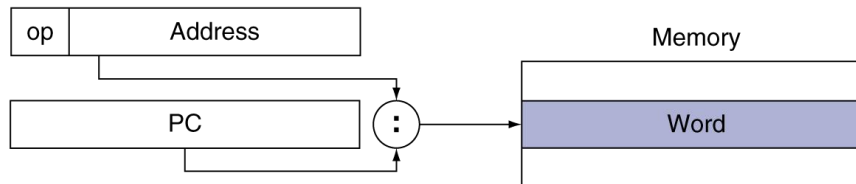
`lw $t0, 0($t1)`

## 4. PC-relative addressing



`bne $s0,$s1, L2`

## 5. Pseudodirect addressing



`j L1`

# Synchronization

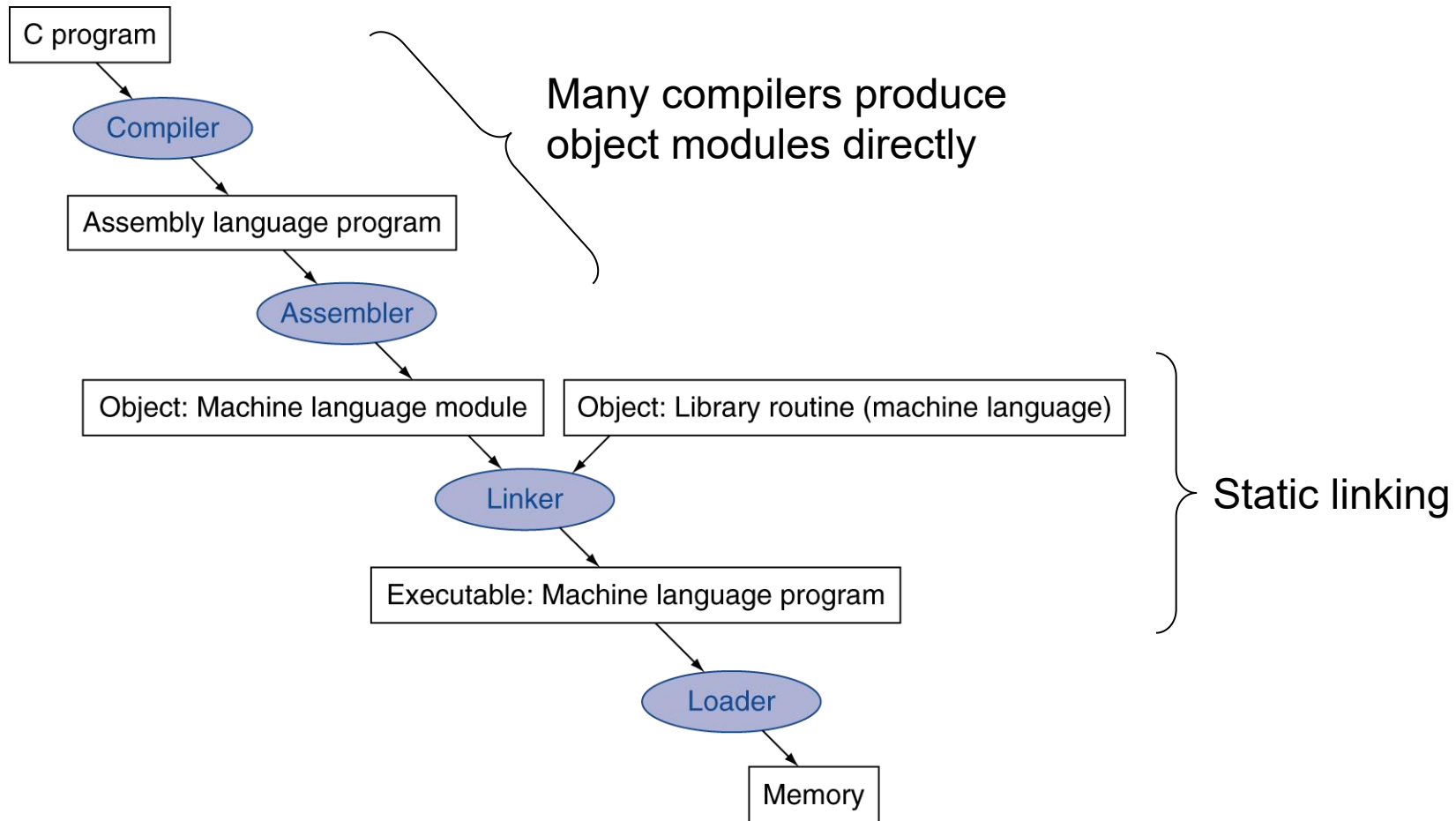
- Two processors sharing an area of memory
  - P1 writes, then P2 reads
  - Data race if P1 and P2 don't synchronize
    - Result depends of order of accesses
- Hardware support required
  - Atomic read/write memory operation
  - No other access to the location allowed between the read and write
- Could be a single instruction
  - E.g., atomic swap of register  $\leftrightarrow$  memory
  - Or an atomic pair of instructions

# Synchronization in MIPS

- Load linked: `ll rt, offset(rs)`
- Store conditional: `sc rt, offset(rs)`
  - Succeeds if location not changed since the `ll`
    - Returns 1 in `rt`
  - Fails if location is changed
    - Returns 0 in `rt`
- Example: atomic swap (to test/set lock variable)

```
try: add $t0,$zero,$s4 ;copy exchange value
 ll $t1,0($s1) ;load linked
 sc $t0,0($s1) ;store conditional
 beq $t0,$zero,try ;branch store fails
 add $s4,$zero,$t1 ;put load value in $s4
```

# Translation and Startup



# Assembler Pseudoinstructions

- Most assembler instructions represent machine instructions one-to-one
- Pseudoinstructions: figments of the assembler's imagination

`move $t0, $t1`       $\rightarrow$    `add $t0, $zero, $t1`

`blt $t0, $t1, L`     $\rightarrow$    `slt $at, $t0, $t1`  
                              `bne $at, $zero, L`

- `$at` (register 1): assembler temporary



# Producing an Object Module

- Assembler (or compiler) translates program into machine instructions
- Provides information for building a complete program from the pieces
  - Header: described contents of object module
  - Text segment: translated instructions
  - Static data segment: data allocated for the life of the program
  - Relocation info: for contents that depend on absolute location of loaded program
  - Symbol table: global definitions and external refs
  - Debug info: for associating with source code

# Linking Object Modules

- Produces an executable image
  1. Merges segments
  2. Resolve labels (determine their addresses)
  3. Patch location-dependent and external refs
- Could leave location dependencies for fixing by a relocating loader
  - But with virtual memory, no need to do this
  - Program can be loaded into absolute location in virtual memory space

# Loading a Program

- Load from image file on disk into memory
  1. Read header to determine segment sizes
  2. Create virtual address space
  3. Copy text and initialized data into memory
    - Or set page table entries so they can be faulted in
  4. Set up arguments on stack
  5. Initialize registers (including \$sp, \$fp, \$gp)
  6. Jump to startup routine
    - Copies arguments to \$a0, ... and calls main
    - When main returns, do exit syscall

# Dynamic Linking

- Only **link/load** library procedure when it is called
  - Requires procedure code to be relocatable
  - Avoids image bloat caused by static linking of all (transitively) referenced libraries
  - Automatically picks up new library versions

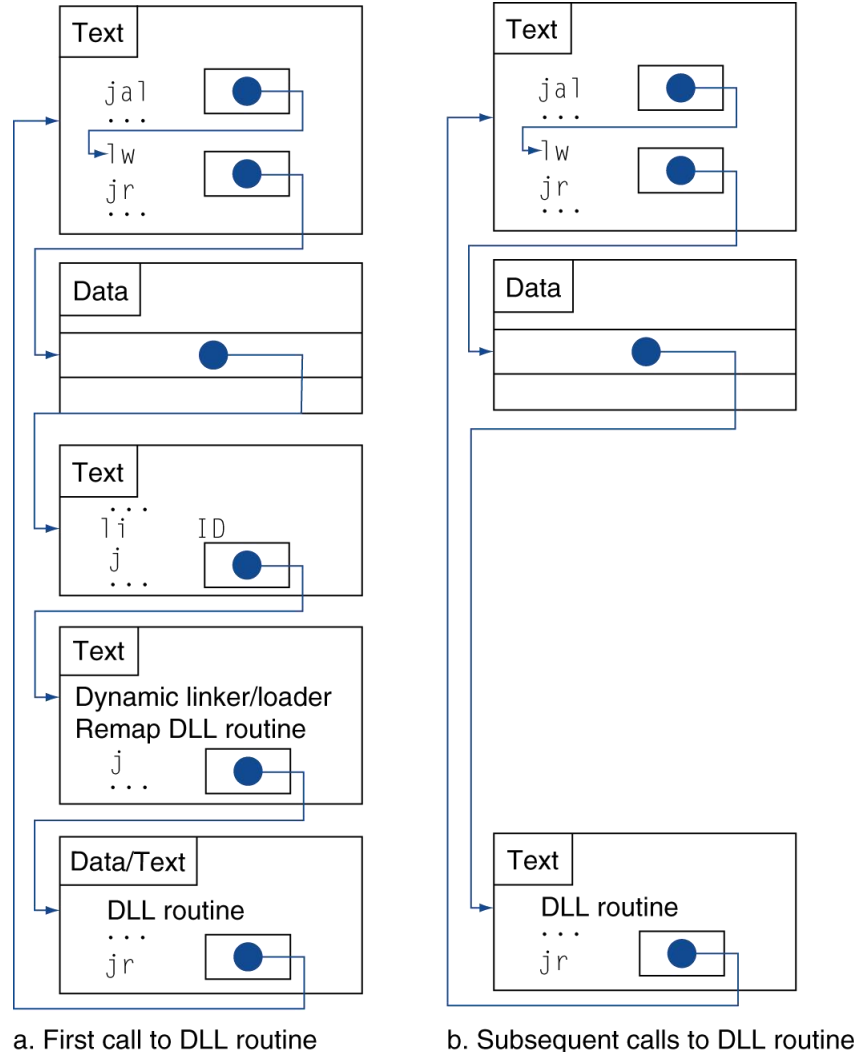
# Lazy Linkage

Indirection table

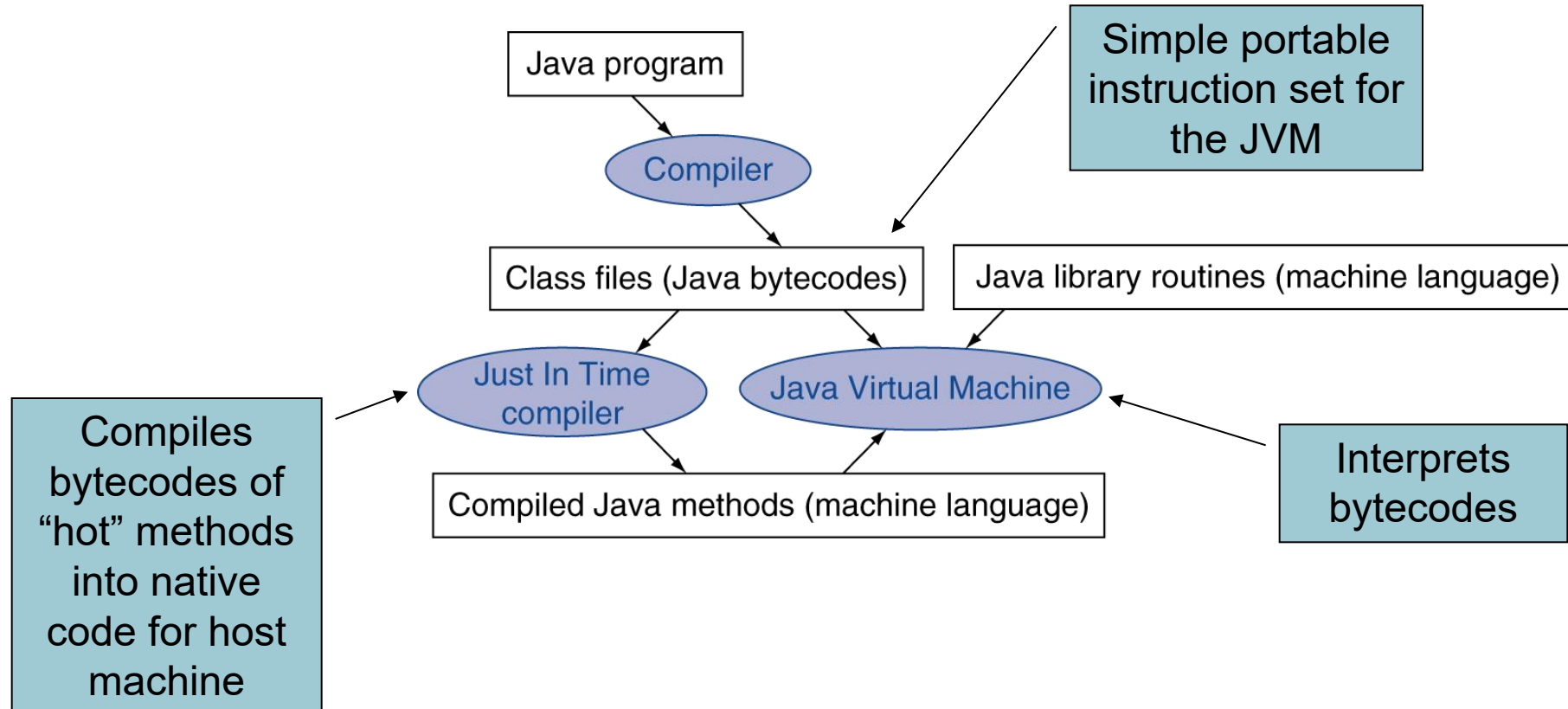
Stub: Loads routine ID,  
Jump to linker/loader

Linker/loader code

Dynamically  
mapped code



# Starting Java Applications



# C Sort Example

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
void swap(int v[], int k)
{
 int temp;
 temp = v[k];
 v[k] = v[k+1];
 v[k+1] = temp;
}
```

- v in \$a0, k in \$a1, temp in \$t0

# The Procedure Swap

|       |                      |                             |
|-------|----------------------|-----------------------------|
| swap: | sll \$t1, \$a1, 2    | # \$t1 = k * 4              |
|       | add \$t1, \$a0, \$t1 | # \$t1 = v+(k*4)            |
|       |                      | # (address of v[k])         |
|       | lw \$t0, 0(\$t1)     | # \$t0 (temp) = v[k]        |
|       | lw \$t2, 4(\$t1)     | # \$t2 = v[k+1]             |
|       | sw \$t2, 0(\$t1)     | # v[k] = \$t2 (v[k+1])      |
|       | sw \$t0, 4(\$t1)     | # v[k+1] = \$t0 (temp)      |
|       | jr \$ra              | # return to calling routine |



# The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (int v[], int n)
{
 int i, j;
 for (i = 0; i < n; i += 1) {
 for (j = i - 1;
 j >= 0 && v[j] > v[j + 1];
 j -= 1) {
 swap(v, j);
 }
 }
}
```

- v in \$a0, k in \$a1, i in \$s0, j in \$s1

# The Procedure Body

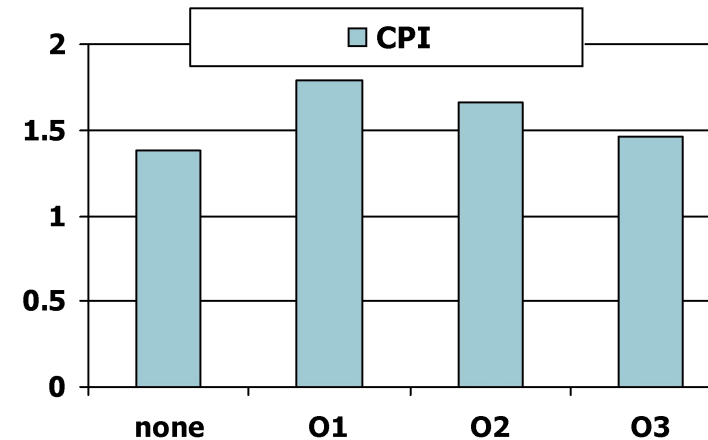
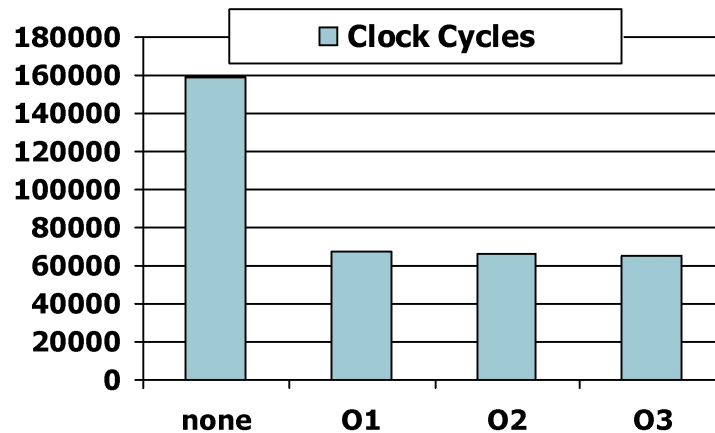
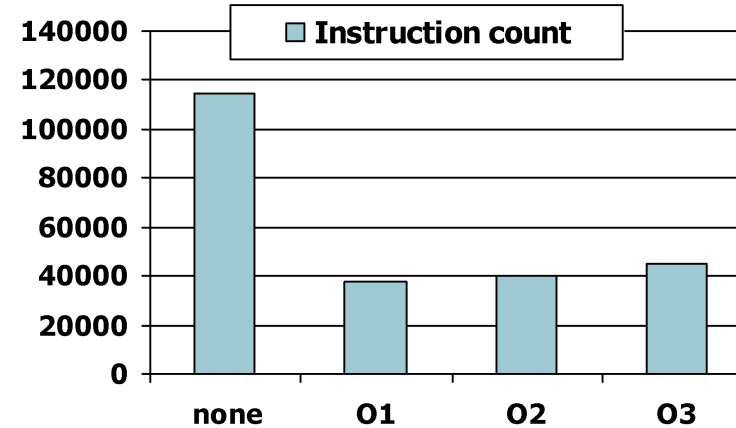
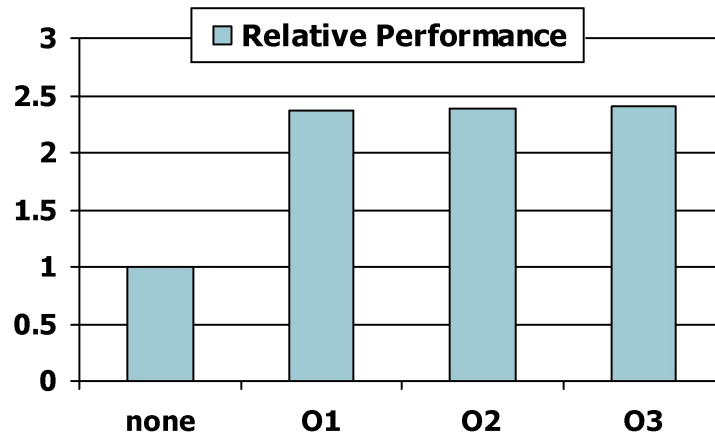
|          |                         |                                      |                          |
|----------|-------------------------|--------------------------------------|--------------------------|
|          | move \$s2, \$a0         | # save \$a0 into \$s2                | Move<br>params           |
|          | move \$s3, \$a1         | # save \$a1 into \$s3                |                          |
| for1tst: | move \$s0, \$zero       | # i = 0                              | Outer loop               |
|          | slt \$t0, \$s0, \$s3    | # \$t0 = 0 if \$s0 ≥ \$s3 (i ≥ n)    |                          |
| for2tst: | beq \$t0, \$zero, exit1 | # go to exit1 if \$s0 ≥ \$s3 (i ≥ n) | Inner loop               |
|          | addi \$s1, \$s0, -1     | # j = i - 1                          |                          |
|          | slti \$t0, \$s1, 0      | # \$t0 = 1 if \$s1 < 0 (j < 0)       |                          |
|          | bne \$t0, \$zero, exit2 | # go to exit2 if \$s1 < 0 (j < 0)    |                          |
|          | sll \$t1, \$s1, 2       | # \$t1 = j * 4                       |                          |
|          | add \$t2, \$s2, \$t1    | # \$t2 = v + (j * 4)                 |                          |
|          | lw \$t3, 0(\$t2)        | # \$t3 = v[j]                        |                          |
|          | lw \$t4, 4(\$t2)        | # \$t4 = v[j + 1]                    |                          |
|          | slt \$t0, \$t4, \$t3    | # \$t0 = 0 if \$t4 ≥ \$t3            |                          |
|          | beq \$t0, \$zero, exit2 | # go to exit2 if \$t4 ≥ \$t3         | Pass<br>params<br>& call |
|          | move \$a0, \$s2         | # 1st param of swap is v (old \$a0)  |                          |
|          | move \$a1, \$s1         | # 2nd param of swap is j             | Inner loop               |
|          | jal swap                | # call swap procedure                |                          |
|          | addi \$s1, \$s1, -1     | # j -= 1                             | Outer loop               |
|          | j for2tst               | # jump to test of inner loop         |                          |
| exit2:   | addi \$s0, \$s0, 1      | # i += 1                             | Outer loop               |
|          | j for1tst               | # jump to test of outer loop         |                          |

# The Full Procedure

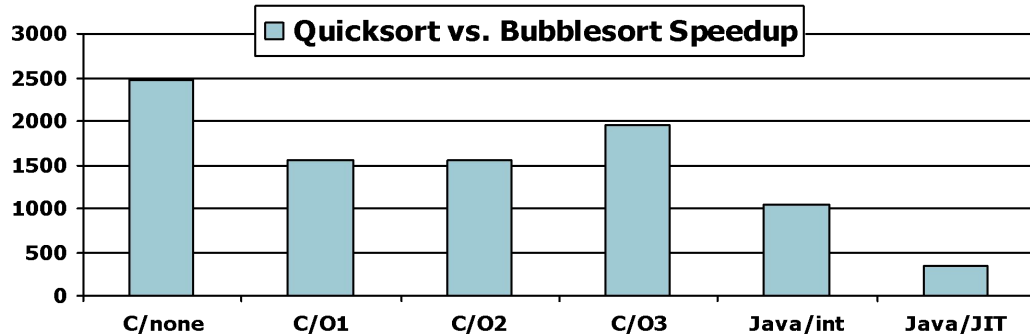
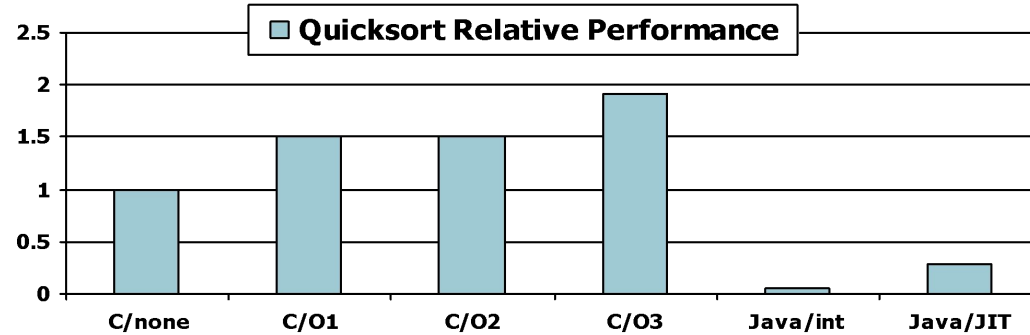
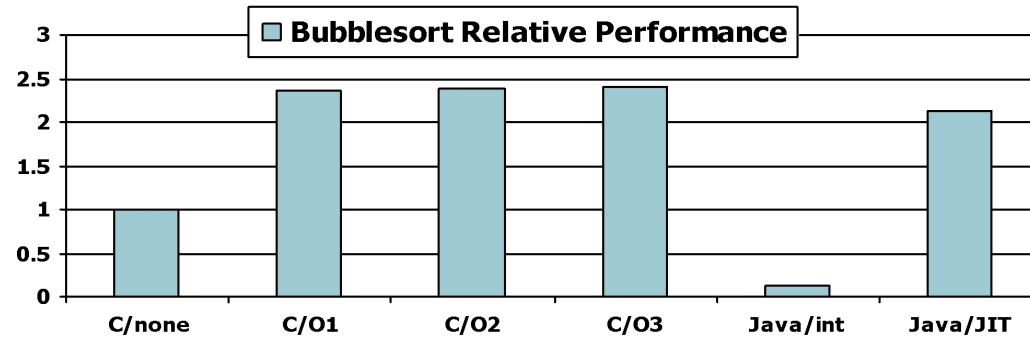
|        |                     |                                      |
|--------|---------------------|--------------------------------------|
| sort:  | addi \$sp,\$sp, -20 | # make room on stack for 5 registers |
|        | sw \$ra, 16(\$sp)   | # save \$ra on stack                 |
|        | sw \$s3,12(\$sp)    | # save \$s3 on stack                 |
|        | sw \$s2, 8(\$sp)    | # save \$s2 on stack                 |
|        | sw \$s1, 4(\$sp)    | # save \$s1 on stack                 |
|        | sw \$s0, 0(\$sp)    | # save \$s0 on stack                 |
|        | ...                 | # procedure body                     |
|        | ...                 |                                      |
| exit1: | lw \$s0, 0(\$sp)    | # restore \$s0 from stack            |
|        | lw \$s1, 4(\$sp)    | # restore \$s1 from stack            |
|        | lw \$s2, 8(\$sp)    | # restore \$s2 from stack            |
|        | lw \$s3,12(\$sp)    | # restore \$s3 from stack            |
|        | lw \$ra,16(\$sp)    | # restore \$ra from stack            |
|        | addi \$sp,\$sp, 20  | # restore stack pointer              |
|        | jr \$ra             | # return to calling routine          |

# Effect of Compiler Optimization

Compiled with gcc for Pentium 4 under Linux



# Effect of Language and Algorithm



# Lessons Learnt

- Instruction count and CPI are not good performance indicators in isolation
- Compiler optimizations are sensitive to the algorithm
- Java/JIT compiled code is significantly faster than JVM interpreted
  - Comparable to optimized C in some cases
- Nothing can fix a dumb algorithm!

# Arrays vs. Pointers

- Array indexing involves
  - Multiplying index by element size
  - Adding to array base address
- Pointers correspond directly to memory addresses
  - Can avoid indexing complexity

# Example: Clearing and Array

```
clear1(int array[], int size) {
 int i;
 for (i = 0; i < size; i += 1)
 array[i] = 0;
}
```

```
 move $t0,$zero # i = 0
loop1: sll $t1,$t0,2 # $t1 = i * 4
 add $t2,$a0,$t1 # $t2 =
 # &array[i]
 sw $zero, 0($t2) # array[i] = 0
 addi $t0,$t0,1 # i = i + 1
 slt $t3,$t0,$a1 # $t3 =
 # (i < size)
 bne $t3,$zero,loop1 # if (...)
 # goto loop1
```

```
clear2(int *array, int size) {
 int *p;
 for (p = &array[0]; p < &array[size];
 p = p + 1)
 *p = 0;
}
```

```
 move $t0,$a0 # p = & array[0]
 sll $t1,$a1,2 # $t1 = size * 4
 add $t2,$a0,$t1 # $t2 =
 # &array[size]
loop2: sw $zero,0($t0) # Memory[p] = 0
 addi $t0,$t0,4 # p = p + 4
 slt $t3,$t0,$t2 # $t3 =
 # (p < &array[size])
 bne $t3,$zero,loop2 # if (...)
 # goto loop2
```



# Comparison of Array vs. Ptr

- Multiply “strength reduced” to shift
- Array version requires shift to be inside loop
  - Part of index calculation for incremented  $i$
  - c.f. incrementing pointer
- Compiler can achieve same effect as manual use of pointers
  - Induction variable elimination
  - Better to make program clearer and safer

# ARM & MIPS Similarities

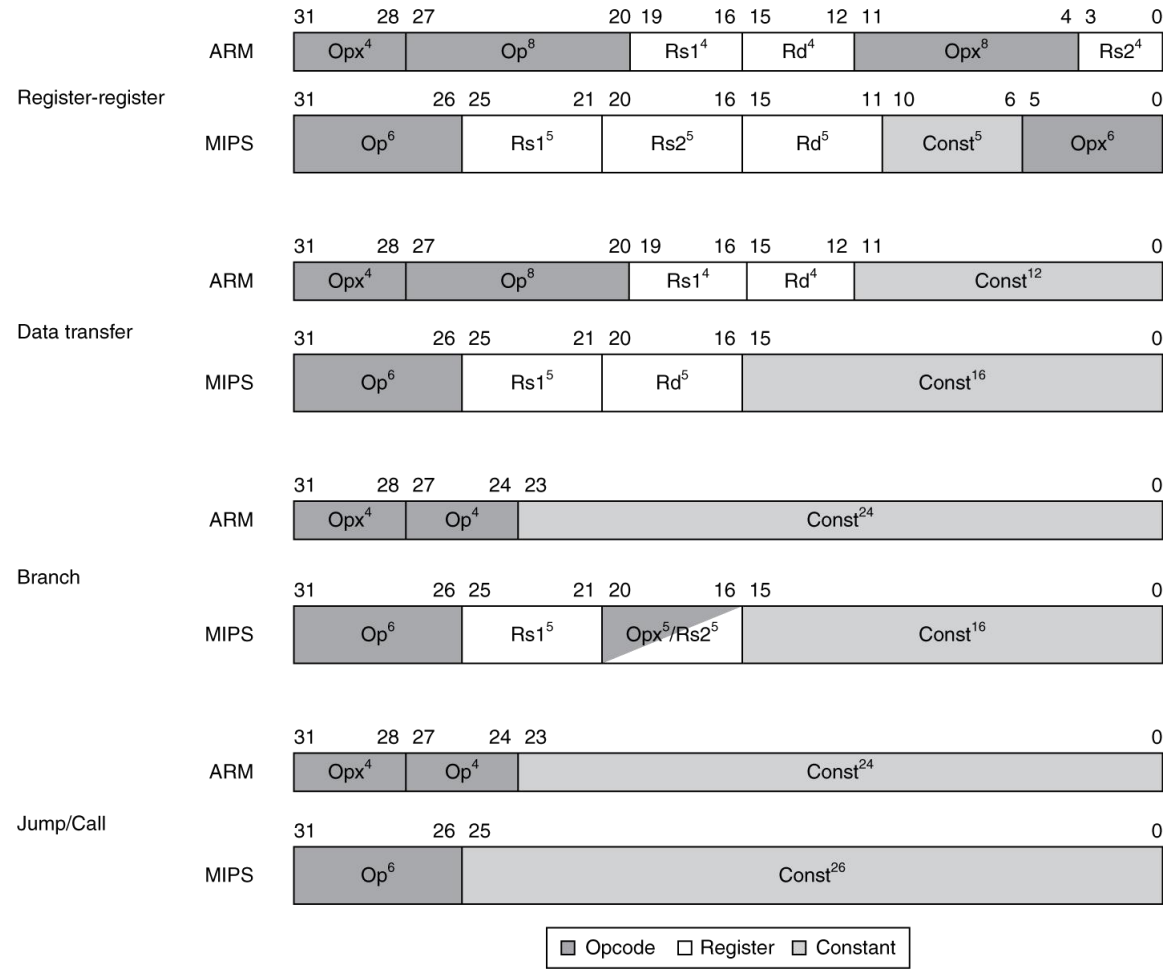
- ARM: the most popular embedded core
- Similar basic set of instructions to MIPS

|                       | ARM           | MIPS          |
|-----------------------|---------------|---------------|
| Date announced        | 1985          | 1985          |
| Instruction size      | 32 bits       | 32 bits       |
| Address space         | 32-bit flat   | 32-bit flat   |
| Data alignment        | Aligned       | Aligned       |
| Data addressing modes | 9             | 3             |
| Registers             | 15 × 32-bit   | 31 × 32-bit   |
| Input/output          | Memory mapped | Memory mapped |

# Compare and Branch in ARM

- Uses condition codes for result of an arithmetic/logical instruction
  - Negative, zero, carry, overflow
  - Compare instructions to set condition codes without keeping the result
- Each instruction can be conditional
  - Top 4 bits of instruction word: condition value
  - Can avoid branches over single instructions

# Instruction Encoding



# ARM v8 Instructions

- In moving to 64-bit, ARM did a complete overhaul
- ARM v8 resembles MIPS
  - Changes from v7:
    - No conditional execution field
    - Immediate field is 12-bit constant
    - Dropped load/store multiple
    - PC is no longer a GPR
    - GPR set expanded to 32
    - Addressing modes work for all word sizes
    - Divide instruction
    - Branch if equal/branch if not equal instructions

# The Intel x86 ISA

- Evolution with backward compatibility
  - 8080 (1974): 8-bit microprocessor
    - Accumulator, plus 3 index-register pairs
  - 8086 (1978): 16-bit extension to 8080
    - Complex instruction set (CISC)
  - 8087 (1980): floating-point coprocessor
    - Adds FP instructions and register stack
  - 80286 (1982): 24-bit addresses, MMU
    - Segmented memory mapping and protection
  - 80386 (1985): 32-bit extension (now IA-32)
    - Additional addressing modes and operations
    - Paged memory mapping as well as segments

# The Intel x86 ISA

- Further evolution...
  - i486 (1989): pipelined, on-chip caches and FPU
    - Compatible competitors: AMD, Cyrix, ...
  - Pentium (1993): superscalar, 64-bit datapath
    - Later versions added MMX (Multi-Media eXtension) instructions
    - The infamous FDIV bug
  - Pentium Pro (1995), Pentium II (1997)
    - New microarchitecture (see Colwell, *The Pentium Chronicles*)
  - Pentium III (1999)
    - Added SSE (Streaming SIMD Extensions) and associated registers
  - Pentium 4 (2001)
    - New microarchitecture
    - Added SSE2 instructions

# The Intel x86 ISA

- And further...
  - AMD64 (2003): extended architecture to 64 bits
  - EM64T – Extended Memory 64 Technology (2004)
    - AMD64 adopted by Intel (with refinements)
    - Added SSE3 instructions
  - Intel Core (2006)
    - Added SSE4 instructions, virtual machine support
  - AMD64 (announced 2007): SSE5 instructions
    - Intel declined to follow, instead...
  - Advanced Vector Extension (announced 2008)
    - Longer SSE registers, more instructions
- If Intel didn't extend with compatibility, its competitors would!
  - Technical elegance ≠ market success



# Basic x86 Registers

| Name   | 31 | 0 | Use                                  |
|--------|----|---|--------------------------------------|
| EAX    |    |   | GPR 0                                |
| ECX    |    |   | GPR 1                                |
| EDX    |    |   | GPR 2                                |
| EBX    |    |   | GPR 3                                |
| ESP    |    |   | GPR 4                                |
| EBP    |    |   | GPR 5                                |
| ESI    |    |   | GPR 6                                |
| EDI    |    |   | GPR 7                                |
|        | CS |   | Code segment pointer                 |
|        | SS |   | Stack segment pointer (top of stack) |
|        | DS |   | Data segment pointer 0               |
|        | ES |   | Data segment pointer 1               |
|        | FS |   | Data segment pointer 2               |
|        | GS |   | Data segment pointer 3               |
| EIP    |    |   | Instruction pointer (PC)             |
| EFLAGS |    |   | Condition codes                      |

# Basic x86 Addressing Modes

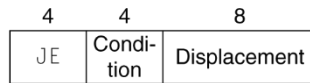
- Two operands per instruction

| Source/dest operand | Second source operand |
|---------------------|-----------------------|
| Register            | Register              |
| Register            | Immediate             |
| Register            | Memory                |
| Memory              | Register              |
| Memory              | Immediate             |

- Memory addressing modes
  - Address in register
  - $\text{Address} = R_{\text{base}} + \text{displacement}$
  - $\text{Address} = R_{\text{base}} + 2^{\text{scale}} \times R_{\text{index}}$  (scale = 0, 1, 2, or 3)
  - $\text{Address} = R_{\text{base}} + 2^{\text{scale}} \times R_{\text{index}} + \text{displacement}$

# x86 Instruction Encoding

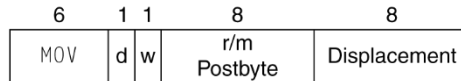
a. JE EIP + displacement



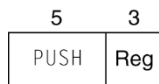
b. CALL



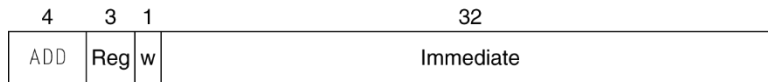
c. MOV EBX, [EDI + 45]



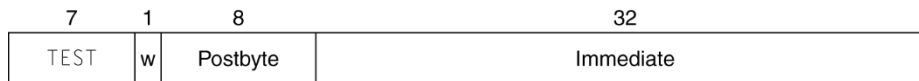
d. PUSH ESI



e. ADD EAX, #6765



f. TEST EDX, #42



- Variable length encoding
  - Postfix bytes specify addressing mode
  - Prefix bytes modify operation
    - Operand length, repetition, locking, ...

# Implementing IA-32

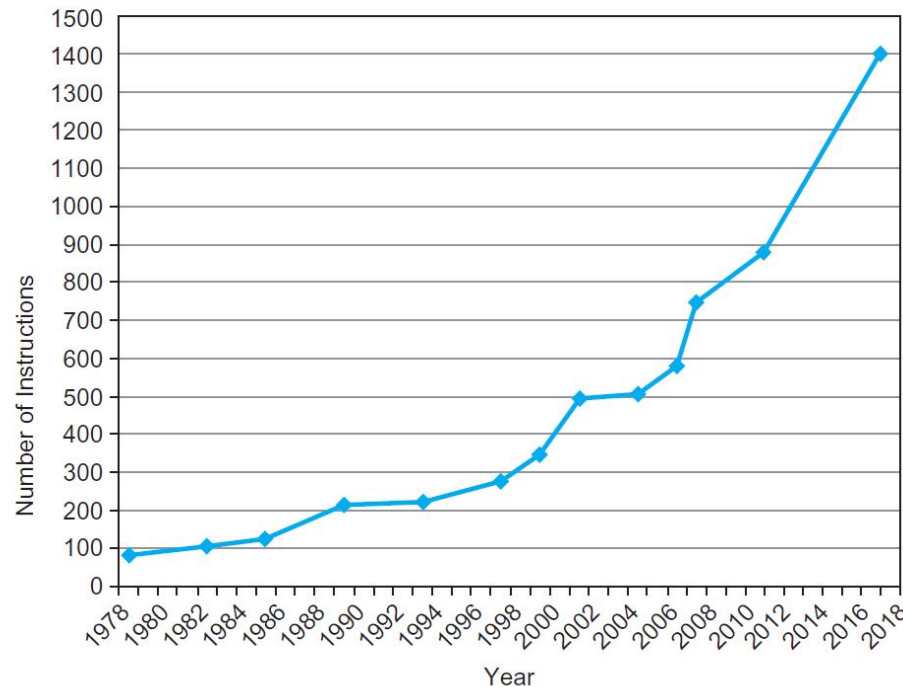
- Complex instruction set makes implementation difficult
  - Hardware translates instructions to simpler microoperations
    - Simple instructions: 1–1
    - Complex instructions: 1–many
  - Microengine similar to RISC
  - Market share makes this economically viable
- Comparable performance to RISC
  - Compilers avoid complex instructions

# Fallacies

- Powerful instruction  $\Rightarrow$  higher performance
  - Fewer instructions required
  - But complex instructions are hard to implement
    - May slow down all instructions, including simple ones
  - Compilers are good at making fast code from simple instructions
- Use assembly code for high performance
  - But modern compilers are better at dealing with modern processors
  - More lines of code  $\Rightarrow$  more errors and less productivity

# Fallacies

- Backward compatibility  $\Rightarrow$  instruction set doesn't change
  - But they do accrete more instructions



x86 instruction set

# Pitfalls

- Sequential words are not at sequential addresses
  - Increment by 4, not by 1!
- Keeping a pointer to an automatic variable after procedure returns
  - e.g., passing pointer back via an argument
  - Pointer becomes invalid when stack popped

# Concluding Remarks

- Design principles
  1. Simplicity favors regularity
  2. Smaller is faster
  3. Make the common case fast
  4. Good design demands good compromises
- Layers of software/hardware
  - Compiler, assembler, hardware
- MIPS: typical of RISC ISAs
  - c.f. x86



# Concluding Remarks

- Measure MIPS instruction executions in benchmark programs
  - Consider making the common case fast
  - Consider compromises

| Instruction class | MIPS examples                        | SPEC2006 Int | SPEC2006 FP |
|-------------------|--------------------------------------|--------------|-------------|
| Arithmetic        | add, sub, addi                       | 16%          | 48%         |
| Data transfer     | lw, sw, lb, lbu, lh,<br>lhu, sb, lui | 35%          | 36%         |
| Logical           | and, or, nor, andi,<br>ori, sll, srl | 12%          | 4%          |
| Cond. Branch      | beq, bne, slt, slti,<br>sltiu        | 34%          | 8%          |
| Jump              | j, jr, jal                           | 2%           | 0%          |